

VISUALIZATION OF DISTRIBUTED ALGORITHMS

Bachelor's Thesis

by

Jannik Koch

November 30, 2025

University of Kaiserslautern-Landau
Department of Computer Science
67663 Kaiserslautern
Germany

Examiner: Prof. Dr. Annette Bieniusa
Philipp Lersch, MSc.

Eigenständigkeitserklärung

Hiermit versichere ich, dass ich die von mir vorgelegte Arbeit mit dem Thema „Visualization of Distributed Algorithms“ selbstständig verfasst habe, dass ich die verwendeten Quellen und Hilfsmittel vollständig angegeben habe und dass ich die Stellen der Arbeit — einschließlich Tabellen und Abbildungen —, die anderen Werken oder dem Internet im Wortlaut oder dem Sinn nach entnommen sind unter Angabe der Quelle als Entlehnung kenntlich gemacht habe.

Kaiserslautern, den 30.11.2025

Jannik Koch

Abstract

Conflict-free Replicated Data Types (CRDTs) have gained significant attention in recent years due to their ability to handle distributed data without immediate synchronization. However, despite this interest, beginner-friendly visualizations and interactive simulations of CRDTs remain limited. This thesis addresses this gap by developing a tool designed to improve understanding of CRDTs through various visualization styles and interactive features. The tool provides multiple visualizations to enhance comprehension of distributed algorithms, interactive simulations in multi-node networks, and pre-scripted, interactive explanations. Users are also able to create custom scenarios using a simple JSON-based scripting language. The system, which builds upon an existing CRDT simulation, incorporates a modular design to allow for easy expansion with new visualization types and CRDTs. A series of use cases were evaluated to ensure the system's effectiveness for small-scale simulations, though it is not intended for large distributed systems. Future enhancements could enable the visualization of broader algorithm categories.

Summary

In recent years, Conflict-free Replicated Data Structures (CRDTs) have become a topic of interest in research and industry. CRDTs permit the execution of operations on local data structures without immediate synchronization under the assumption that operations will be propagated to the rest of the network at some point in the future—a concept known as *strong eventual consistency* [2, 33].

Despite the increased interest in CRDTs, beginner-friendly visual explanations and interactive simulations remain sparse. CRDTs are primarily described through textual descriptions and pseudocode, which creates barriers to entry for beginners interested in the topic.

In this work, a variety of visualization styles capable of improving understanding of distributed algorithms, including CRDTs, are explored, evaluated, and elaborated on.

The result is the creation of a tool that enables users to improve their understanding of CRDTs, with the following capabilities:

1. Providing a variety of visualizations that enhance understanding of distributed algorithms, particularly CRDTs.
2. Offering interactive simulations of CRDTs in a multi-node network, allowing users to explore their behavior, advantages, and disadvantages.
3. Delivering pre-scripted, interactive explanations for CRDTs and enabling users to create their own network scenarios using a simple scripting language.

The tool developed in this thesis is based on an existing CRDT simulation [21]. This thesis outlines the requirements, design decisions, and implementation details for the tool. The original system was restructured with high modifiability in mind, allowing for the easy addition of new visualization types, CRDTs, and interactive explanations. Key changes in the system include the decoupling of frontend and backend, the development of a visualization component capable of rendering multiple views simultaneously, the introduction of a scripting component for creating interactive explanations, and the refactoring and implementation of various CRDTs. Using this restructured system, new visualizations, CRDTs, and interactive explanations were integrated. Furthermore, users were empowered to create their own interactive explanations via a simple JSON-based scripting language.

To evaluate the system, a range of use case scenarios that users may find relevant were successfully addressed. The evaluation included direct user interaction with CRDTs in a network simulation, the creation of user-defined scripts, and navigation through a pre-scripted showcase. In its current form, the system is well-suited for small-scale network simulations, offering user-friendly explanations and visualizations for CRDTs. However, it is not designed for, nor capable of, simulating large distributed systems.

The system’s modular architecture allows for easy future expansions, including the addition of new visualization styles, CRDT types, and interactive network scenarios. Larger-scale architectural changes could enable the visualization of new algorithm categories, such as broadcast algorithms.

Contents

1. Introduction	1
2. Background	3
2.1. What are CRDTs?	3
2.1.1. State-based CRDTs	4
2.1.2. Operation-based CRDTs	4
2.1.3. Delta CRDTs	5
2.2. Semantics of CRDTs	5
2.2.1. Counters	5
2.2.2. Registers	6
2.2.3. Sets	7
2.2.4. Other CRDTs and real-world application	10
3. Related Work	11
3.1. Space-Time Diagrams	11
3.1.1. Logging-based Space-Time Diagrams	12
3.2. Live Message Passing	12
3.2.1. <i>CRDT Showcase</i> : A live CRDT Network Simulation	14
3.3. Abstract Executions	16
3.4. Semilattices	17
3.5. Visualization of States	17
4. Design and Implementation	19
4.1. Requirements	19
4.2. Evaluation and Choice of Visualization Types	21
4.2.1. Network graph with Live Message Travel	22
4.2.2. Space-Time Diagram	22
4.2.3. Semilattice	25
4.2.4. Abstract Executions	27
4.2.5. Choice of Visualization Styles	28
4.3. Design and Implementation of Visualizations	30
4.3.1. Architectural Changes	30
4.3.2. Implementation of Space-Time View	31
4.3.3. Implementation of Meshing	32
4.4. Design and Implementation of Scripting Component	32
4.4.1. Script Structure	32
4.4.2. Showcases	36
4.4.3. Scripting architecture	36
4.4.4. Supported Operations	38

4.5. Design and Implementation of Data Structures and Node Visualizations	41
4.5.1. The Strategy Abstract Class	43
4.5.2. CRDT types	44
5. Evaluation and Discussion	47
5.1. Architecture Evaluation	47
5.2. Qualitative Performance Evaluation	48
5.2.1. Complete Network separation	48
5.2.2. Operation vs. State-based CRDTs in Unreliable Networks	49
5.2.3. CRDT Introduction	49
5.3. Visualization Style Evaluation	50
5.4. Quantitative Performance Evaluation	50
5.5. Limitations	51
5.5.1. Node Size	51
5.5.2. Viability of Large Networks	52
5.6. Future Work	52
6. Conclusion	55
List of Figures	57
A. Appendix	59
Bibliography	65

1. Introduction

Conflict-free Replicated Data Types (CRDTs) have become a prominent focus in the design of distributed systems, attracting attention across industry 2.2.4, academia [2, 27, 33], and education. These data structures enable eventual consistency and high availability, making them well-suited to modern distributed systems, collaboration tools, and decentralized applications [32, 26, 34, 31, 7]. Despite their increased rate of use, the understanding of CRDTs may be limited by the lack of intuitive and accessible dynamic visualizations.

Currently, most explanations of CRDTs rely heavily on textual descriptions and algorithmic representations, sometimes accompanied by static graphics [2, 33, 27, 29, 30]. While these resources are valuable for a detailed technical understanding, formal specification, and eventual implementation, they may not be sufficient for newcomers or those looking for a more approachable introduction to the concept. The relative absence of dynamic, real-time visualizations besides the examples [21, 20] provided in this thesis can hinder initial comprehension and engagement, particularly for individuals with limited exposure to the theoretical aspects of distributed systems.

A dynamic visual approach explaining CRDTs could simplify their complexities and spark greater interest, providing a more accessible way for beginners to explore their capabilities. By offering a visual explanation, it becomes easier to grasp the core principles behind CRDTs, and their advantages and limitations can be demonstrated more clearly. Furthermore, such visual tools could serve as an educational resource, helping individuals and teams to experiment with and better understand the behavior of CRDTs in a variety of scenarios.

Given the potential benefits of visual explanations, this paper explores the following research questions:

1. How can the various algorithms, especially CRDTs, for distributed systems be effectively and interactively visualized?
2. In what ways can the advantages, disadvantages, capabilities, and limitations of CRDTs be represented visually?
3. How can a user-friendly and extensible tool providing interactive visualizations of distributed algorithms be realized?
4. How can users be empowered to create customized network scenarios and observe CRDT behavior in those environments?

To create an overview of the topic, this thesis extensively introduces Conflict-free Replicated Data Types, places them in the context of other distributed

algorithms, and explains some of their advantages and disadvantages. Different classes of CRDTs are explained with regard to their update propagation mechanisms and the type of their underlying data structure.

Furthermore, extant literature is surveyed to create an overview of different visualization styles used for the purpose of explaining distributed algorithms in general and CRDTs in particular. The surveyed visualization styles are evaluated and extended to methodically create a set of visualization styles best-suited to visualizing CRDTs.

The objective of this thesis is the documentation of the design, implementation, and evaluation of a tool capable of addressing the above research questions. Specifically, the tool should be capable of providing interactive CRDTs in a network simulation. The collected visualization styles can then be used to improve understanding of the network's and data structure's current and past states, as well as their behaviors. Users should be able to interact with the system by creating and deleting nodes, altering network settings, and executing operations on CRDTs via a Graphical User Interface. To further improve understanding of CRDTs, a mechanism enabling programmers and users with the ability to create interactive network scenarios with accompanying text-based explanations shall be designed implemented. The system shall be constructed in a modular manner, allowing for easy addition of new CRDTs, visualization styles, and scripts.

In the final evaluation, the tool's ability to explain various network scenarios and CRDT behaviors via manual user input, pre-scripted interactive explanations, and user-created scripts shall be validated, accompanied by an assessment of the tool's modifiability.

2. Background

2.1. What are CRDTs?

Distributed systems face a variety of challenges with regard to data consistency. The CAP theorem states that any distributed system may only provide guarantees for two of the three properties [9, 15]:

1. Consistency: For a system to be consistent, "there must exist total order on all operations such that each operation looks as if it were completed at a single instant"[15]. Operations on different nodes must be serializable.
2. Availability: Every request to a node that is not failing must yield a response [15].
3. Partition-tolerance: The system's is able to return valid responses even if a component of the network is entirely separated from the rest of the network. A dropped message can be interpreted as a short-lived system partition [15].

Conflict-free Replicated Data Types attempt to address this problem by relying on strong eventual consistency. CRDTs are defined by the following properties:

- (i) A replica can be modified independently without coordinating with other replicas.
- (ii) By adopting mathematically sound rules guaranteeing state convergence, any two replicas that have received the same set of updates reach the same state [28, 27].

As a consequence, CRDTs are not dependent on constant, unimpeded network access. Instead, local offline changes can be made, and replica convergence is achieved in an asynchronous manner under the assumption that updates are delivered to all nodes eventually. Executing operations without prior synchronization or data fetching also enables low latencies independent of network conditions. By design, consensus is not required [33]. In terms of the CAP theorem, high availability and partition-tolerance are prioritized over consistency. Typically, CRDTs provide a well-defined API which allows the user to access the underlying data structure, often a set, counter, or register (see Section 2.2). This allows programmers to import a CRDT library without implementing consistency-generating code on-the-fly [2, 28].

2.1.1. State-based CRDTs

State-based CRDTs transmit the entirety of their current state to other nodes in the network. These nodes then merge the received state with their current state by applying a merge function. According to Preguiça *et al.* [33][27], the states of all nodes converge if:

- (i) An update causes the state of a node to inflate, where its state is larger or the same size as before the merge operation.
- (ii) The merge function produces the least upper bound $s_1 \sqcup s_2$ of two states s_1, s_2 .
- (iii) All possible states of a node are partially ordered and form a semilattice.

The merge function must be both idempotent and commutative to guarantee eventual convergence. As a result, state-based CRDTs are capable of functioning in networks with weak guarantees and will converge so long as any new state eventually reaches all nodes. Duplicated messages are resolved due to the merge function's idempotence, out-of-order delivery is resolved by its commutativity, and the information contained in dropped messages can be re-transmitted when a node broadcasts its entire state after its next update [33, 2, 27, 28].

One disadvantage of this approach is the potential size of messages, all of which are required to transport the entire state of the replicate data structure. An increment-only counter, for example, stores a distinct value for every node in the network [33, 11]. This can partly be ameliorated by not sending a state-containing message after every update; instead, we transmit the state only after a few operations have been performed which will still guarantee eventual convergence due to the merge function's properties. This lightens network load but comes at the price of slightly decreased consistency.

Depending on their specific implementation, state-based CRDTs may become inefficient as time progresses due to inflating states and unbalanced data structures. The need for garbage collection may add additional complexity [33, 29].

2.1.2. Operation-based CRDTs

In contrast, operation-based CRDTs only send the operations they have applied to themselves. Any node that receives such a message from another node applies the same operation to itself, altering its own state. If a node fails to receive a message containing an operation, its own state will irrecoverably diverge from the sender node. Due to this, operation-based CRDTs require additional network guarantees when compared to state-based CRDTs, namely algorithms capable of ensuring that an operation is reliably and exactly-once propagated to every node in the network. Operation-based CRDTs may also specify a delivery order as a requirement for the network, usually a causal order [27, 33, 2].

The need for an exactly-once delivery is a consequence of the potential non-idempotence of operations, since a duplication of, for example, an increment operation will lead to state divergence. Likewise, non-commutative operations sent from the same node – such as removing from and adding to a set – require causal delivery which results in additional overhead in the messages containing the operation [2].

Since operation-based CRDTs do not transmit their entire state, network load is lower in comparison to state-based CRDTs. This does not, however, stop the states of operation-based replicas from potentially inflating.

2.1.3. Delta CRDTs

Delta CRDTs attempt to combine the advantages of state-based and operation-based CRDTs. Instead of propagating their entire state, they propagate the difference between a node's state before and after one or more updates [3, 2].

2.2. Semantics of CRDTs

A large variety of CRDTs with different concurrency semantics have been developed. The simplest CRDTs are usually based on sets, counters and registers [33, 2]. Some operations on these data structures, such as *add* operations on a set or *increment* operations on a counter, are commutative and can be easily incorporated into CRDTs. In data structures where operations do not commute, e.g. *add* and *remove* operations in a set [33], some viable approaches to make states converge are:

1. Add-Wins: *Add* operations take precedence over concurrent *remove* operations.
2. Remove-Wins: In a remove-wins semantic, a *delete* operation takes precedence over an *add* operation.
3. Last-Writer-Wins: In an LWW semantic, all updates are subject to a total order in which later updates with a higher timestamp prevail over earlier updates [2, 27].

2.2.1. Counters

Counter-CRDTs store one or more integer(s) that can be incremented and/or decremented, depending on the specific algorithm. Increments and decrements only increase or decrease the integer by a single unit, respectively. A Counter-CRDT converges to the number of all executed increments in a system minus the number of all executed decrements [27, 28, 33].

One such CRDT is a state-based increment-only vector counter. This counter stores a separate counter for every individual node as a vector. To obtain a single value from this, a node adds up all individual values in the vector. An increment operation on a node only increments the value of the vector element

corresponding to that node. After a successful increment, the node sends out an update containing its new state to the network. If a node receives a state, it will compare its own vector with the received vector and choose the maximum of each entry for its new state. The algorithm for an increment-only vector counter is shown in Listing 2.1: [33]

Listing 2.1: *A class implementing a state-based Vector-Counter CRDT.*

```
class GrowOnlyCounterNode{

    int[n] C

    constructor(){
        this.C = [0, 0, ... , 0]
    }

    query(){
        return ( $\sum_{i \in C} i$ )
    }

    increment(){
        C[myID]++
        broadcast(C)
    }

    merge(receivedPayload: int[n]){
        for (int j = 0; j < C.size; j++){
            C.j = max(receivedPayload.j, C.j)
        }
    }

}
```

This counter can easily be modified to support decrement operations by adding a second vector N that counts decrements. The query function would then return the sum of the increment P vector minus the sum of the decrement vector N :

$$(\sum_{i \in P} i) - (\sum_{i \in N} i)$$

This is called a PN-counter [33, 2].

2.2.2. Registers

A register is capable of storing a value and supports the operations *write(value)* and *read()* [2, 27, 28, 33].

One Register CRDT is the LWW Register in which all updates are assigned a timestamp. Timestamps are considered unique and totally ordered, the total order being consistent with causal order. Early implementations reminiscent of this data structure date back to the 1970s [22]. LWW Registers can be implemented as both state-based and operation-based CRDTs. Any *write* operation updates state by assigning a new value and generating a new timestamp. If an update is received, its timestamp is compared to the current timestamp and the register takes the value associated with the higher timestamp [33].

Another register-based CRDT is the Multi-Value Register. A Multi-Value Register stores all concurrent write events in a set E which is subject to a partial order $\leq$. A query on a Multi-Value Register will return any element that does not have another element partially-ordered above them: [2, 33, 37, 11]

$$Query() = \{v | \exists a \in E : a = write(v) \wedge \neg \exists b \in E : b = write(else) \wedge a \leq b\}$$

2.2.3. Sets

Set-based CRDTs support an *add* operation and may also support a *remove* operation. *Add* and *remove* operations are not commutative. Consider the following sequence of operations executed on a *NodeA*: $add(a), add(a), remove(a)$.

The resulting state of *NodeA* will be the empty set $\{\}$, since $add(a)$ is idempotent. Each operation triggers the transmission of a message containing a copy of *NodeA*'s state to *NodeB*. However, the messages travel at different speeds and are delivered out of order. *NodeB* receives the sequence $\{a\}, \{\}, \{a\}$ because the message triggered by the second $add(a)$ operation travelled slower than the message triggered by the $remove(a)$ operation. As a result, *NodeB* includes $\{a\}$ in its internal state. The replicas have become inconsistent even though both of them were aware of the same updates, violating the definition of CRDTs [see 2.1].

A variety of set-based CRDTs can remedy this. One of the simplest approaches is to forbid *remove* operations. Adding elements to a set is:

- (i) Commutative: For any set S and two successive add-operations $add(a), add(b)$ for any elements a, b , it holds that:

$$\text{First op.: } add(a), \text{ second op.: } add(b) \implies S' = S \cup \{a, b\}$$

and also

$$\text{First op.: } add(b), \text{ second op.: } add(c) \implies S' = S \cup \{a, b\}$$

- (ii) Idempotent: For a set S and an operation $add(a)$ for any element a , any application of $add(a)$ beyond the first application does not change the result. For example:

$$S.add(a) = S.add(a).add(a) = S \cup \{a\}$$

- (iii) Associative: For a set S' and three successive operations $add(a)$, $add(b)$, $add(c)$, where operations in brackets are executed before operations outside brackets, it holds that:

$$(S'.add(a), S'.add(b)), S'.add(c) \implies S'' = S' \cup \{a, b, c\}$$

and

$$S'.add(a), (S'.add(b), S'.add(c)) \implies S'' = S' \cup \{a, b, c\}$$

From these properties, it is easy to derive a merge function that satisfies the idempotence and commutativity requirements of a state-based CRDT by creating the union of any two states. The result is a grow-only-set state-based CRDT, shown in Listing 2.2:

Listing 2.2: *An implementation of a grow-only set.*

```
class GrowOnlyNode{

    Set S

    constructor(){
        S = {}
    }

    query{
        return S
    }

    add(element e){
        S = S ∪ {e}
        broadcast(S)
    }

    merge(Set receivedS){
        S = S ∪ receivedS
    }
}
```

One method of implementing a remove operation is the creation of an additional tombstone set R . Elements may be moved from the live set S to the tombstone set, but not the other way around. Ergo, once an element has been deleted from S , it may not be re-added to S . *Remove* operations only add an element to R if the element is currently contained in S . This is called a Two Phase Set (2P Set). In a state-based 2P Set as seen in Listing 2.3, the *merge* operation creates the unions $S' \cup S$ and $R' \cup R$ of the live sets and the tombstone sets [33, 2].

Listing 2.3: *An implementation of a 2P-Set.*

```
class 2PSetNode{

    Set S
    Set R

    constructor(){
        S = {}
        R = {}
    }

    query(element e){
        return (e ∈ S && e ∉ R)
    }

    add(element e){
        if (e ∉ R){
            S = S ∪ {e}
            broadcast(S,R)
        }
    }

    remove(element e){
        if (e ∈ S){
            S = S \ {e}
            R = R ∪ {e}
            broadcast(S,R)
        }
    }

    merge(Set receivedS, Set receivedR){
        R = R ∪ receivedR
        S = (S ∪ receivedS) \ R
    }
}
```

These are only two among many other examples of Set-CRDTs that have been proposed. Other examples for set-based CRDTs include:

- (i) Operation-based 2P-Set: An operation-based implementation of the 2P set [33].
- (ii) U-Set: This modification of the 2P-Set is based on two assumptions: Firstly, elements are unique and may not be re-added after being removed. Secondly, a causal delivery algorithm ensures that an operation $add(e)$ arrives before an operation $remove(e)$ for the same element. [33]

- (iii) LWW Element Set: This CRDT is based on the last-write-wins semantic. In case of concurrent *add* and *remove* updates, whichever update is totally-ordered after the other takes precedence [27, 33].
- (iv) Observed-remove Set / Add-wins Set: In this set, *add* operations take precedence over concurrent *remove* operations of the same element. Each element receives a unique tag not visible to the user on creation. If the same element is added twice, it will be stored as two distinct $(element, ID)$ tuples, but the user will only see the element once in a *query* operation. A *remove*(*e*) operation records all IDs associated with *e* and includes them in the message to be sent to other replicas. On arrival, all elements *e* with an ID included in the message are deleted while elements *e* with an ID not included in the message were written concurrently with the *remove*(*e*) operation and remain in the set [33, 2, 27, 6].

2.2.4. Other CRDTs and real-world application

A multitude of CRDTs beside variations of the three basic types described above have been proposed. Tree-based and sequence CRDTs find application both in collaborative text editing [29, 23, 30] and in collaborative JSON editing [12].

Since their inception, CRDTs have become a widely-used technology, successfully applied, among others, in the following:

1. Offline edits on Apple's Notes app synchronize using CRDTs [32].
2. Soundcloud developed a LWW CRDT named *Roshi* which supports *insert* and *delete* operations [7].
3. Facebook uses CRDTs to manage consistency for online access to the Facebook graph [34].
4. *Bet365* uses Riak's implementation of the OR-Set CRDT [26].
5. *League of Legends* uses a CRDT provided by Riak for its chat system [31].
6. *Yjs* is a JavaScript library providing a variety of data types to facilitate collaborative editing [17].

3. Related Work

A variety of visualizations exist for technologies in the domain of distributed systems in general and fewer for CRDTs specifically. To find a suitable visualization style for distributed systems, especially a visualization style suitable for CRDTs, a variety of visualizations from various sources are collected and described in this chapter. The visualizations collected will be expanded on and evaluated with regard to their suitability to visualizing CRDTs in Section 4.2.

3.1. Space-Time Diagrams

Space-time diagrams are one of the most widespread visualizations of distributed systems, especially when small-scale interactions between few nodes are concerned. Space-time diagrams are utilized, among others, to visualize CRDTs [27, 33, 28, 2] and Broadcast algorithms [13, 14, 18].

They consist of one or more timelines, each representing a single node. On these timelines are placed events which denote either a local operation, a send event or a receive event. Events may be annotated by various descriptions. Forward-facing arrows from one timeline to another represent messages between replicas. One example for a typical space-time diagram is Figure 3.1.

Space-time diagrams are capable of modelling a happens-before order, defined as follows:

An event e_1 happened-before e_2 ($e_1 \prec e_2$) if:

- (i) e_1 occurred before e_2 in the same process (here: on the same node).
- (ii) Or if e_1 is a *send* event and e_2 is the corresponding *receive* event.
- (iii) Or if there is an event e' with $e_1 \prec e'$ and $e' \prec e_2$ [19].

In a CRDT, $e_1 \prec e_2$ means that the effects of e_1 have been applied at the node executing e_2 [27].

Space-time diagrams visualize a happens-before order if we assume that:

1. Any event e_1 on a timeline is visible to any event e_2 occurring later on the same timeline, so $e_1 \prec e_2$.
2. Arrows are not permitted to travel backwards, i.e. if timeline A has a send event a_1 which is received as b_1 in timeline B , then any send event b_2 on B with $b_1 \prec b_2$ is unable to travel back in time to arrive at a point before a_1 occurred.

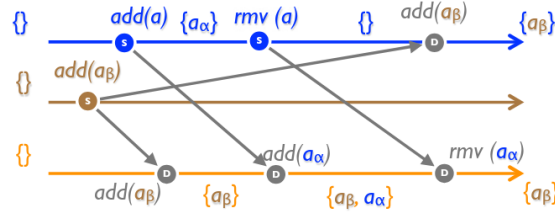


Figure 3.1.: A space-time diagram from Preguiça et al. [33] used to showcase an Observed-Remove Set.

3.1.1. Logging-based Space-Time Diagrams

A variety of works utilize space-time diagrams to showcase the behavior of distributed systems for logging purposes. One such example is ShiViz [1, 5], a utility that reconstructs interactive space-time diagrams from logs. The logs are parsed by user-defined regular expressions, which allows ShiViz to accommodate a variety of logging formats. Real-time message passing and dynamic behavior is not modelled. Instead, diagrams are only generated after a set of logs has been obtained. The resulting interactive space-time diagrams are displayed next to the logs they were obtained from, as is shown in Figure 3.2. The lines between two events from different timelines represent a happens-before relation between them. Hovering over a log entry causes the corresponding event in the space-time diagram to be highlighted and vice versa [1, 5].

Another application, Falcon [24], utilizes a similar approach. Falcon is capable of constructing a happens-before order from multiple logs by means of a constraint solver. Falcon, like ShiViz, requires the collection of logging data and then reconstructs the order of events afterwards instead of constructing a space-time diagram in real-time. The space-time diagram is not generated dynamically [24].

3.2. Live Message Passing

Dynamic visualizations of distributed algorithms can be used to showcase the algorithm's behavior in action.

One visualization of the Raft consensus algorithm, published on GitHub [25], sends heartbeat and leader-election messages back and forth between multiple nodes. The nodes themselves portray their own state by various means:

- (i) A color coding for each node represents their current election period.
- (ii) A circular loading bar represents the time remaining to the next leader election.
- (iii) A set of dots equal to the number of nodes represents how many votes a node received in a leader election.

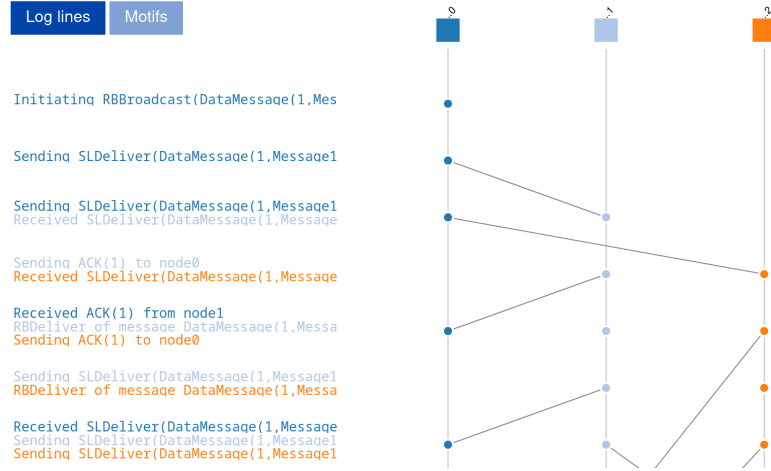


Figure 3.2.: *ShiViz [5] places the space-time diagram next the logs it was reconstructed from.*

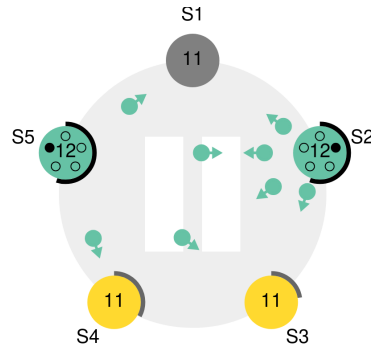


Figure 3.3.: *The Raft visualization from [25] passes live messages and displays the system's current global state.*

Crucially, the state of all nodes and messages is visible at all times, enabling any observer to view the system's global state. This visualization provides an additional leader history chart which provides information on the system's history, but the message passing visualization by itself as shown in Figure 3.3 does not provide such capabilities.

Another interactive visualization of CRDTs has been created by Jake Lazaroff [20]. In this visualization, two nodes are placed side by side and the user can directly alter their state depending on which CRDT they utilize. Once the user has altered a node's state, an animated message may be sent to its partner node, depending on the specific example. The user is capable of setting a message delay and switching the connection between the nodes on and off entirely. These visualizations are interspersed throughout a text-based explanation of CRDTs and various code snippets, as is shown in Figure 3.4. Note that this visualization only simulates a network of two nodes at most. Fur-

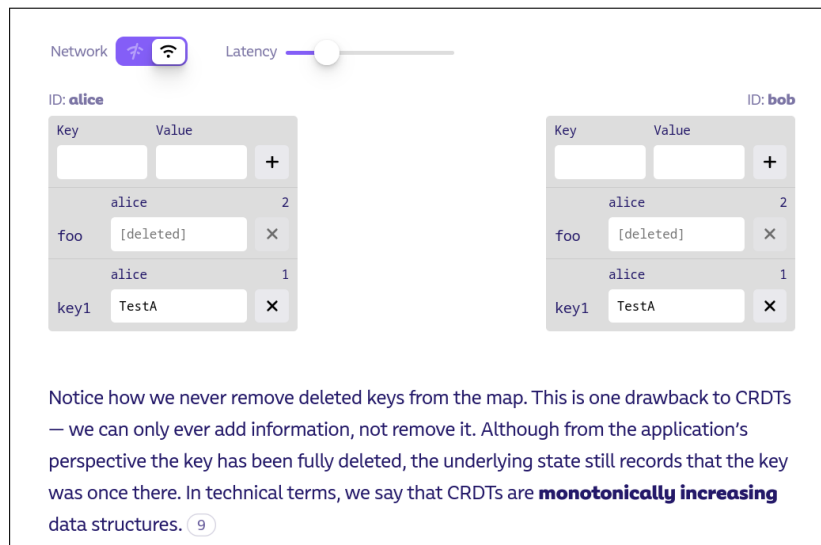


Figure 3.4.: This visualization mixes text-based explanations with live message passing, but only shows two nodes [20].

thermore, the system's history is not visualized. Like [25], the system's global state is visible to the user at all times.

3.2.1. CRDT Showcase: A live CRDT Network Simulation

The *CRDT Showcase* by Philipp Lersch, shown in Figure 3.5 is a real-time simulation of CRDTs and other data structures in a network [21]. The user can choose a data structure and create multiple fully-meshed nodes, which can be toggled offline and online. The nodes display a name, the data structure's operations, and the node's state. The user can interact with the nodes by accessing the data structure's operations. If the user performs an operation on one of the nodes, the node automatically sends a message to all other nodes in the system. This message floats from the sender node to all receiver-nodes in a predefined timeframe. Messages can either successfully arrive at the receiver-node, or duplicate on the way, or be dropped without arriving at the receiver-node.

The user can alter network settings, including the travel time and travel time range of a message, the probability with which a message is dropped, and the probability of message duplication. The system also provides a number of presets which can be selected by the user. If a preset is selected by the user, the system creates a predefined set of nodes of a predefined data structure and sets the network settings to predefined values. As a result, the user is not required to manually set up the entire system on every use.

Since all nodes are meshed in the *CRDT Showcase*, visualizing connections between nodes to show the network's topology in a network graph is redundant.

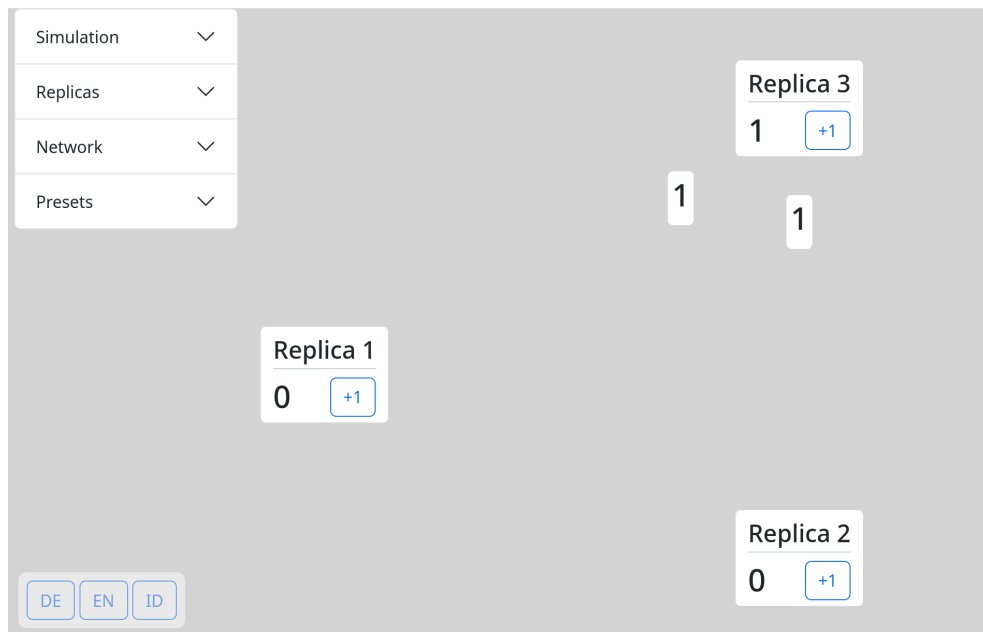


Figure 3.5.: *This instance of the CRDT Showcase is configured to show three nodes with a simple counter data structure. Replica 3 has previously incremented its own counter, resulting in two messages with the content (1) travelling to Replica 1 and Replica 2. Options and settings are located in the top left corner.*

3.3. Abstract Executions

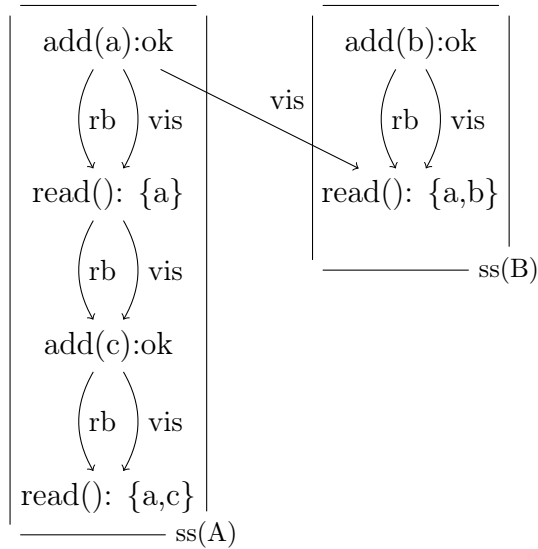
Burckhardt [10] defines an abstract execution as an event graph $(E, op, rval, rb, ss, vis, ar)$ where

- (i) $(E, op, rval, rb, ss)$ is a history H ,
- (ii) vis is an acyclic relation describing the relative timing of update propagation. If $a \xrightarrow{vis} b$, then the effect of a is visible to the client performing b . Two updates are concurrent if they are not ordered by visibility.
- (iii) ar is a total order on all operations that describes how update conflicts are resolved, especially for non-commutative operations.

The history $H = (E, op, rval, rb, ss)$ named in (i) is an event graph, where:

- (i) E is the set of operations performed on clients.
- (ii) op describes the operation of an event.
- (iii) $rval$ describes the return values of operations.
- (iv) rb is a returns-before order on all events in E . It describes the ordering of non-overlapping operations.
- (v) ss , the same-session equivalence relation. It showcases which operations have been performed in the same session.

An abstract execution for a grow-only set, inspired by [10] may look as follows:

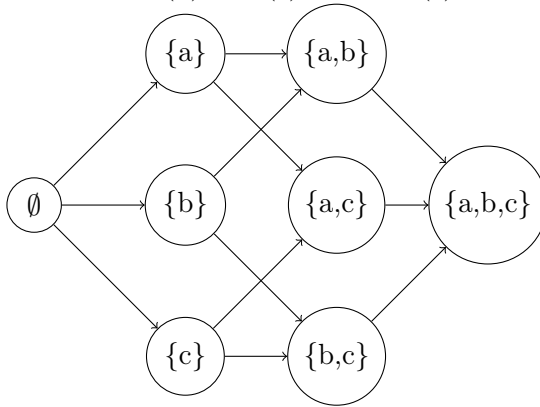


Abstract executions are well-suited to visualizing which operations have become visible to other operations through the *vis* relation. They do not concern

themselves with the states of individual nodes at discrete points in time. Instead, they give an overview over all operations applied in the system, which of these operations are visible to other operations, and how they are arbitrated. [36, 10]

3.4. Semilattices

Some visualizations are more obscure or only applicable to certain types of CRDTs. It is possible to visualize the states of multiple nodes in a state-based CRDT in a partially ordered join semilattice. This ability to form a join semilattice is, in fact, one of the conditions a valid merge function must exhibit [33, 27]. A complete semilattice for a Grow-Only Set-CRDT with the operations $add(a)$, $add(b)$, and $add(c)$ may look as follows:



This visualization in particular is well-suited to showcasing the merge function's commutativity and the eventual convergence of states.

3.5. Visualization of States

The visualization of states is heavily dependent on the underlying data structure. The state of many simpler CRDTs does not necessarily need a novel type of visualization to be easily understood. A Set-CRDT's state, for example, can easily be discerned via a standard set notation, i.e. $S = \{a, b, c\}$. Another example is a vector counter, which can be visualized as a vector $[1, 1, 1]$ for a sample network of three nodes. Registers are equally easy to visualize, Figure 3.4 providing an example for a key-value storage consisting of multiple registers.

Some authors use visualizations to make the semantics of more complicated CRDTs understandable. As an example, Treedoc is a tree-based CRDT and consequently uses trees to visualize the underlying data structure, as shown in Figure 3.6 [29]. A Sequence-CRDT, the *RGA* (Replicated Growable Array), is visualized by its authors as a linked-list of nodes, as can be seen in Figure 3.7 [30].

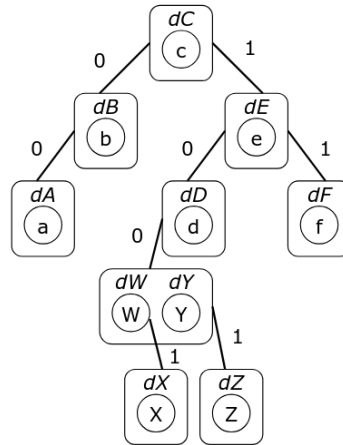


Figure 3.6.: A tree diagram visualizing Treedoc's data structure [29].

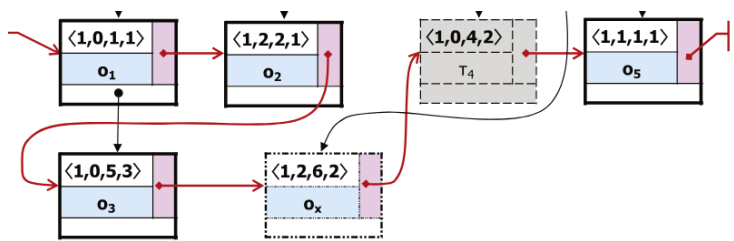


Figure 3.7.: An excerpt from a graphic in [30] visualizes a part of the CRDTs underlying data structure: a linked list.

4. Design and Implementation

In this chapter, the design decisions, architecture decisions, and implementation are described. The three major components to be designed and implemented were

- (i) the visualizations,
- (ii) the scripting component,
- (iii) the data structure component.

4.1. Requirements

To grant a comprehensive understanding of CRDTs, the visualization needs to be comprehensive in its coverage of a network's and its nodes' past and current states. Therefore, the visualization shall fulfill the following requirements:

- /V1/ The visualization shall show the current content of messages and nodes.
 - (a) The system shall visualize the operations and current state of all nodes.
 - (b) In messages of state-based CRDTs, the visualization shall display the state of the node the message originated from.
 - (c) In messages of operation-based CRDTs, the visualization shall display the operation to be executed on arrival.
- /V2/ The visualization shall display the messages travelling between nodes. Dropped and duplicated messages shall be explicitly visualized.
- /V3/ The entire system's history shall be visible. This includes:
 - (a) All operations executed in the past.
 - (b) All *send* and *receive* events that occurred in the system's history.
 - (c) The resulting states of nodes after any past event, including both send and receive events.
- /V4/ The visualization shall visualize the happens-before relation over all nodes.
- /V5/ The visualization shall display the network's topology.

- /V6/ The visualization shall not be convoluted or complex to a degree where users might feel overwhelmed.
 - (a) The number of unique elements in the visualization shall remain low.
 - (b) The number of visible connection lines (arrows, edges, and links) shall remain low. The number of different types of connection lines shall remain low. The number of instances where connection lines cross each other shall remain low.
- /V7/ The visualization of the system's global state shall be applicable to all types of CRDTs.

The simulated system shall support the following use cases:

- /U1/ The user shall have the ability to select pre-scripted Showcases that explain CRDT behavior.
- /U2/ The user shall be able to upload their own scripts and have the system execute them.
- /U3/ The system shall permit users to choose a CRDT of their choice and interact with its API.
- /U4/ The user shall be able to alter the network's properties. This includes the ability to set the probability that a message will be dropped or duplicated, define the message delay as a discrete value or a range, and alter the network's meshing.
- /U5/ The user shall be able to delete and create nodes, as well as toggle them offline or online.
- /U6/ The user shall be able to switch between all provided views via button clicks.

The system's purpose is to make CRDTs, their properties, and their advantages and disadvantages easily understandable to inexperienced users. The mere ability to execute operations on CRDTs in a simulated distributed system may not be sufficiently explanatory without additional explanations of what is happening in the system. Therefore, the system shall be able to execute *showcases*. A showcase has the following properties:

- /S1/ A showcase can be selected by the user via the user interface.
- /S2/ When a showcase is chosen, the entire network is reset and replaced by a network specified in the showcase.
- /S3/ The showcase shall be able to execute pre-defined tasks. This includes the ability to execute operations on the nodes, change network settings, change node settings, change the network's meshing,

and send messages between nodes, all without user input.

- /S4/ The system shall be able to create popup-explanations which explain the pre-defined tasks the system has executed or is about to execute. When a user has read the popup-explanation and is ready to continue the showcase, they can do so by clicking a *continue*-button in the popup.
- /S5/ The user shall be able to define showcases in the program's source code.
- /S6/ The user shall be able to create showcases by means of a scripting language and upload them to the simulation.
- /S7/ While designing a showcase, the user shall be able to define *procedures*. A procedure is a set of operations within a showcase. A procedure is executed by clicking the *continue*-button in the showcase.

The system is intended as an introduction to distributed algorithms in general and CRDTs in specific. A variety of visualizations are viable for this purpose, and a large number of distributed algorithms exist. The system must allow easy use, the addition of new visualization styles, and the addition of new algorithms. Therefore, the following quality measures, paraphrased from the *ISO 25010* standard [16], shall be prioritized during the system's construction:

1. Interaction Capability

- a) Learnability: How quickly and how easily can users learn the system?
- b) Operability: Do the system's attributes make it easy to control?
- c) User engagement: Is the GUI structured to be inviting and manner and does it motivate further interaction with the system?
- d) Self-descriptiveness: Does the system make its capabilities and functions obvious to the user without having to resort to documentation?

2. Maintainability

- a) Modularity: The degree to which the system is comprised of isolated components, the changing of which has minimal impact on other components.
- b) Modifiability: The degree to which the system can be modified without creating new faults [16].

4.2. Evaluation and Choice of Visualization Types

In this section, the visualization styles collected and described in Chapter 3 are

- (i) extended by a variety of features to fulfill a larger number of functional requirements than their original versions previously described, and
- (ii) evaluated with regard to their suitability to visualize CRDTs.

4.2.1. Network graph with Live Message Travel

Visualizing the network as a set of nodes sending live messages back and forth results, as described in Section 3.2.1, has different advantages and disadvantages, depending on implementation. Firstly, a set of nodes can be turned into a network graph by adding edges where connections between nodes exist. Live messages can be propagated along edges after a node performs an operation. The resulting network graph is capable of visualizing the network's topology in an intelligible manner, providing a major advantage over other visualization styles.

A node-based view is well-suited to displaying the system's current state. As is exemplified by Section 3.2.1, all nodes can display their name, current state, and available operations. The user is able to see messages in transit, send events, receive events, duplications, and drop-events at the moment they occur. Messages in transit are capable of displaying either their own state or an operation to be executed on arrival, shown in Figure 3.5. As such the entire system's current state is displayed at all times.

A major disadvantage is the network graph view's inability to display the network's history. Operations, send events, and receive events are not stored. Once a sent message arrives at its destination, the only indication that an operation has been performed is the node's altered state. A node's past states are not stored. The user is incapable of retracing how the system achieved its current state. Similarly, since the node view does not store its history, it is impossible to reconstruct a *happens-before* relation.

Evidently, the node view strongly emphasizes visualizing the system's current state over its history and must be merged with or accompanied by another visualization capable of showing the network's history.

Visual clutter is a problem in two separate situations: Firstly, a network containing a large number of fully meshed nodes results in a lot of edges. A fully-meshed network of six nodes, for example, results in $\left(\sum_{i=1}^5 i\right) = 15$ edges. Secondly, multiple concurrent updates in a fully-meshed network containing a similarly large number of nodes will result in a high number of messages travelling through the network. Both of these disadvantages are displayed in Figure 4.1.

4.2.2. Space-Time Diagram

A major advantage of space-time diagrams is their ability to visualize all events executed in the system, including the relation between all send events and their corresponding receive events. They also visualize a happens-before relation, as described in Section 3.1. Past operations can be displayed by labelling the corresponding events with the description of an operation. A node's state

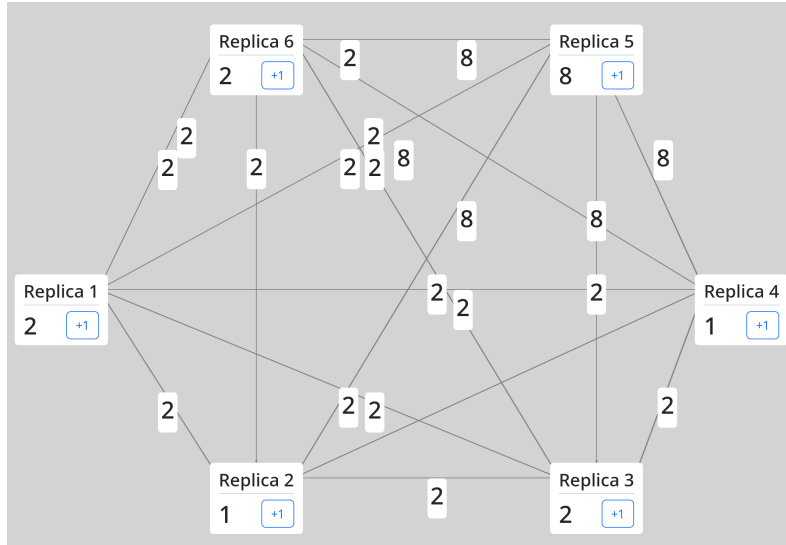


Figure 4.1.: A large number of messages in a fully-meshed network containing as few as six nodes cause significant clutter.

after an operation can be visualized either by hovering over or clicking the corresponding event on the diagram, triggering the appearance of a floating element containing the state. These design choices satisfy all requirements regarding the system's history and allow the user to trace the system's history back in its entirety.

The system's current state, too, can be visualized by space-time diagrams. A single representation of a node containing its state and the CRDT's API, interactive to the user, can be placed at the beginning of a timeline. Any triggered send event can be created on the timeline immediately and spawn a message containing either the origin node's state or an operation to be applied. This message travels to another timeline and creates both a new receive event on arrival and an arrow connecting it to its send event. Operation-based CRDTs can send small messages containing a single operation without causing major overcrowding. State-based CRDTs, however, may suffer from space constraints, as messages contain large states and a significant part of the visualization is reserved for displaying the network's history. Space-time diagrams, therefore, are mostly capable of visualizing the current state of a system. A sample visualization for a space-time diagram with all features listed above is Figure 4.2.

The visualization of meshing is a concern for space-time diagrams. One possibility is the construction of a network graph consisting of the nodes at the top of the timelines and edges connecting them to each other. While this is possible, as seen in Figure 4.4, it would introduce additional complexity into the diagram. Users are forced to disambiguate between three different types of lines: (i) Meshing edges, (ii) timelines, and (iii) message travel paths.

A major disadvantage of space-time diagrams is clutter. Every event is

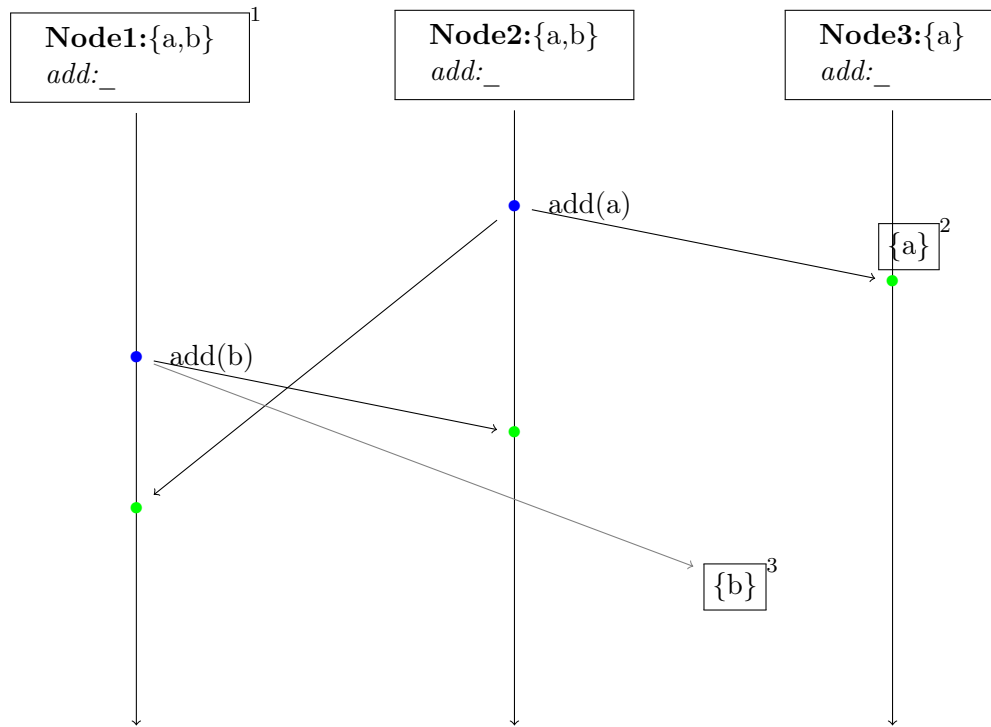


Figure 4.2.: A space-time diagram visualizing a system's history and its current state. A set-based CRDT is used.

¹ A node's current state is displayed above the timeline. Users can create new set elements via the `add: _` field.

² If the user hovers over an event, the node's state at the time of the event is displayed.

³ Messages display their content during transit. Here, the messages contains Node1's state after the operation `add(b)`

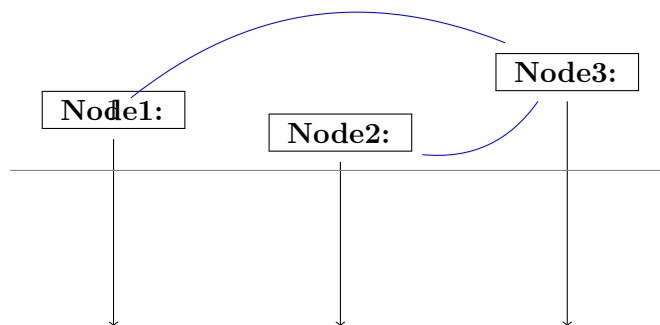


Figure 4.3.: A space-time diagram visualizing meshing. Edges between nodes are colored blue, while the horizontal gray below the nodes separates the nodes from the timelines.

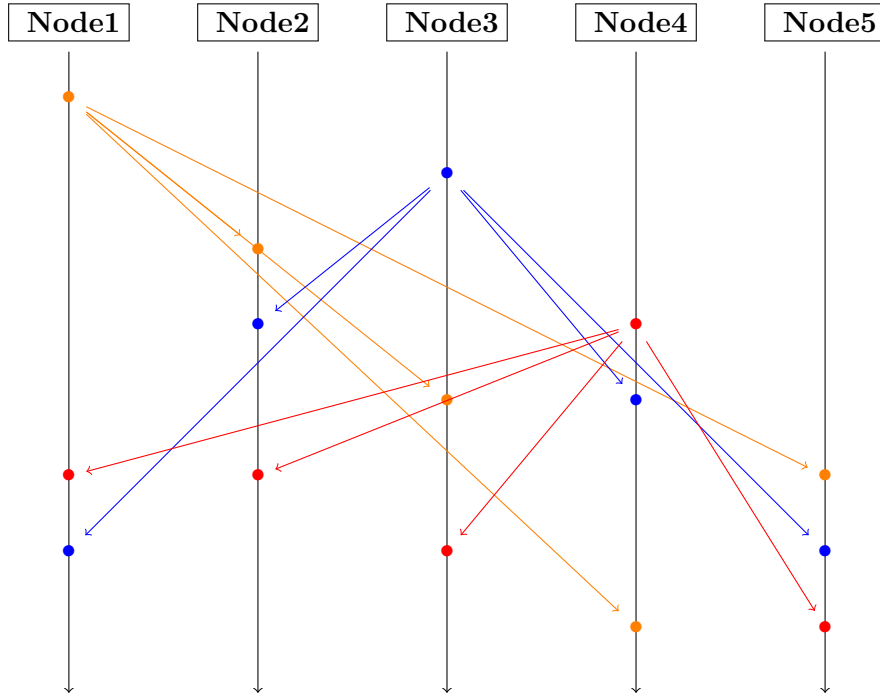


Figure 4.4.: *Despite the diagram’s color-coding, a large number of events and messages can make a space-time diagram confusing to the user.*

visible on the system, usually connected to either send or receive events by an arrow. To exemplify this, we imagine a network of five fully-meshed nodes in which messages are not dropped. If three of these nodes perform concurrent operations and send a message to all four neighboring nodes, the diagram would display a minimum of 15 events and 12 possibly overlapping arrows. The result is a cluttered visualization in which the system’s history is complete but incomprehensible to the user, as can be seen in Figure 4.4.

4.2.3. Semilattice

Semilattices are an interesting and little-used choice for visualizing CRDTs. They do not showcase the system’s topology and heavily deviate from the more familiar space-time diagrams and network-graph views. A semilattice visualization is capable of showing individual replicas travelling through the semilattice until they eventually converge to the same state, under ideal circumstances. They are, therefore, uniquely capable of showcasing the strong eventual consistency typical of CRDTs.

Describing a node’s path through the semilattice can be achieved in various ways. If each node is assigned a color, then their paths through the semilattice can be indicated by adding an arrow of the same color from one state to another whenever a node changes its state. Additionally, the operation performed by the node can be layered on top of the corresponding arrow, which satisfies Requirement 3a. A node’s current state can be visualized by a dot of the

node's assigned color located within the corresponding state of the semilattice. Coloring the corresponding state itself is not viable as multiple nodes can be located inside the same state. An example for a semilattice view exhibiting these features is shown in Figure 4.5.

Despite their advantages, semilattice-based views can have major downsides. Firstly, there is no obvious visualization for live messages between nodes, nor is there an obvious visualization for received messages. Consider the semilattice in Figure 4.5: The red node changes its state from $\{b\}$ to $\{b, c\}$, but the corresponding arrow does not have an operation, implying that the state change was triggered when it received a message from the blue node, sent when it changed from $\{\}$ to $\{c\}$. Labelling the arrow with an annotation that the transition was caused by a receive event is hard to visually distinguish from a normal operation.

It is also difficult to display a CRDT's API. A Register CRDT requires, at the very least, a text input field for its write-operation. This text field would have to be displayed either at all times, in which case a single state in the semilattice could be populated by multiple text input fields, or the text input field would be activated by clicking or hovering on a dot representing the node. Even then, the node might receive a message while a user enters text, so the node would be travelling across the semilattice while the user is trying to interact with it.

Another problem is the potential size a semilattice can reach. In the worst case, a complete semilattice representing a Set-CRDT must contain the powerset $\mathcal{P}(S)$ for the set S of all elements added globally. For just 6 elements, this would amount to a semilattice of $\mathcal{P}(S) = 2^6 = 64$ states.

It is also unclear how semilattices should behave when receiving a message containing a state that has strongly diverged from the receiving node's state. If, for example, a green node joined the network in Figure 4.5 just before the red and blue nodes transitioned from state $\{b, c\}$ to state $\{a, b, c\}$, it would receive a message containing the state $\{a, b, c\}$ and immediately transition from $\emptyset$ to $\{a, b, c\}$. No path through the semilattice accurately describes the green node's state transition since all states differ from their predecessor by only the inclusion of a single element, while the green node added three elements to its state at once.

To rectify some of the above issues concerning the use of semilattices, we replace them with *state graphs*. A state graph is a directed graph where:

1. any state a that has existed in the system at some point in any node is represented by a single vertex $V(a)$ in the graph. There are no vertices for states that no node has ever reached.
2. When a node n transitions from some state a to some state b , an edge in the state graph is created from vertex $V(a)$ to vertex $V(b)$. The edge is marked as belonging to node n .

The resulting state graph in Figure 4.6 is smaller than the corresponding semilattice in Figure 4.5 and permits for more than one concurrent operation to be

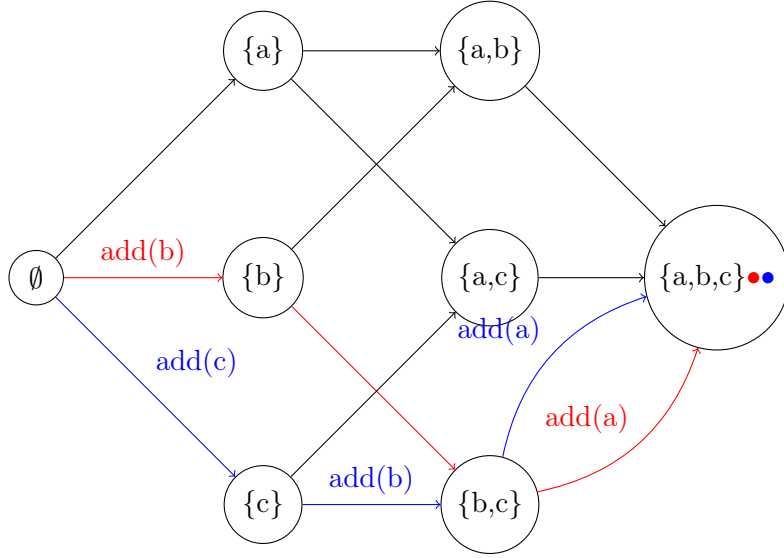


Figure 4.5.

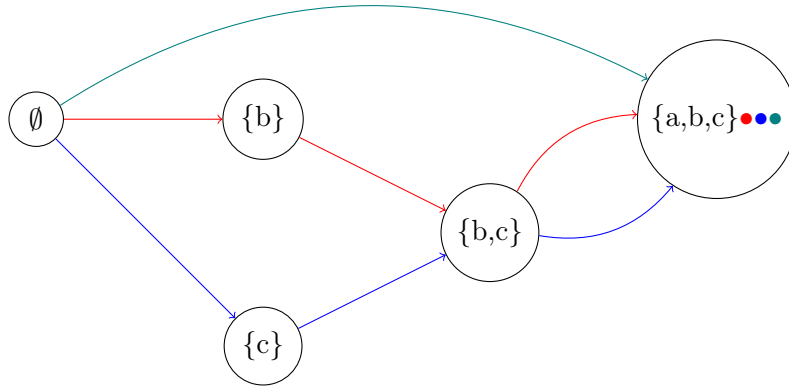


Figure 4.6.: A state graph in which transitions of each node are marked in a unique color.

applied to a node, all the while demonstrating the property of strong eventual consistency.

4.2.4. Abstract Executions

Abstract executions yield an exhaustive visualization with regard to the system's history. They display every operation *op* executed, including its return value *rval*. Abstract executions do not explicitly model nodes, so the flow of messages between nodes is not directly apparent and only implicitly stated in the *vis* relation. Likewise, the state of nodes cannot be displayed; at most, a read operation may be representative of some node's state when it occurs.

A major disadvantage of abstract executions is the necessity for the user to understand the formal definitions integral to abstract executions, including

		Space-Time	Netw. Graph	State graph	Abstr. Ex.
/V1/	Current state	(✓)	✓	(✓)	
/V1/ a)	Nodes	✓	✓	✓	
/V1/ b)	Msg state-based	(✓)	✓		
/V1/ c)	Msg op-based	(✓)	✓		
/V2/	Live msg travel	(✓)	✓		
/V3/	History	✓		(✓)	✓
/V3/ a)	Operations	✓			✓
/V3/ b)	Events	✓			✓
/V3/ c)	States	✓		✓	(✓)
/V4/	Happens-Before	✓			✓
/V5/	Topology	(✓)	✓		
/V6/	Understandability	(✓)	(✓)	✓	(✓)
/V7/	Universality	✓	✓	✓	✓

Table 4.1.: ✓: The requirement is fulfilled.

(✓): The requirement is (i) fulfilled but significantly reduces understandability, or (ii) partly fulfilled.

Blank space: The requirement is not fulfilled.

three different types of relations. This makes abstract executions less suitable as easily understandable visualization.

4.2.5. Choice of Visualization Styles

Each visualization style's ability to fulfill the visualization requirements is evaluated in Section 4.2.5. No single visualization style fulfills all functional requirements to a satisfying degree. Both state graphs and abstract executions fail to fulfill the majority of functional requirements, despite their unique capabilities. Space-time diagrams, when implemented with all additions described above, manage to fulfill every single functional requirement, though not always to a satisfying degree, seeing as live messages and the inclusion of topology decrease the visualization's understandability as defined in Requirement 6. Network graphs, while failing to fulfill every requirement concerning the network's history, fulfill every single requirement concerning the network's current state, live message travel, and topology to a sufficient degree.

If the system provides both a space-time diagram and a network graph to the user in parallel, every single functional requirement is sufficiently fulfilled. To reduce clutter, live messages and network topology are not included in the space-time diagram. Space-time diagrams retain the ability to show states when hovering on events and display performed operations next to events. The result is shown in Figure 4.7. Therefore, the decision was made to use a network-graph and a space-time diagram as parallel visualization styles.

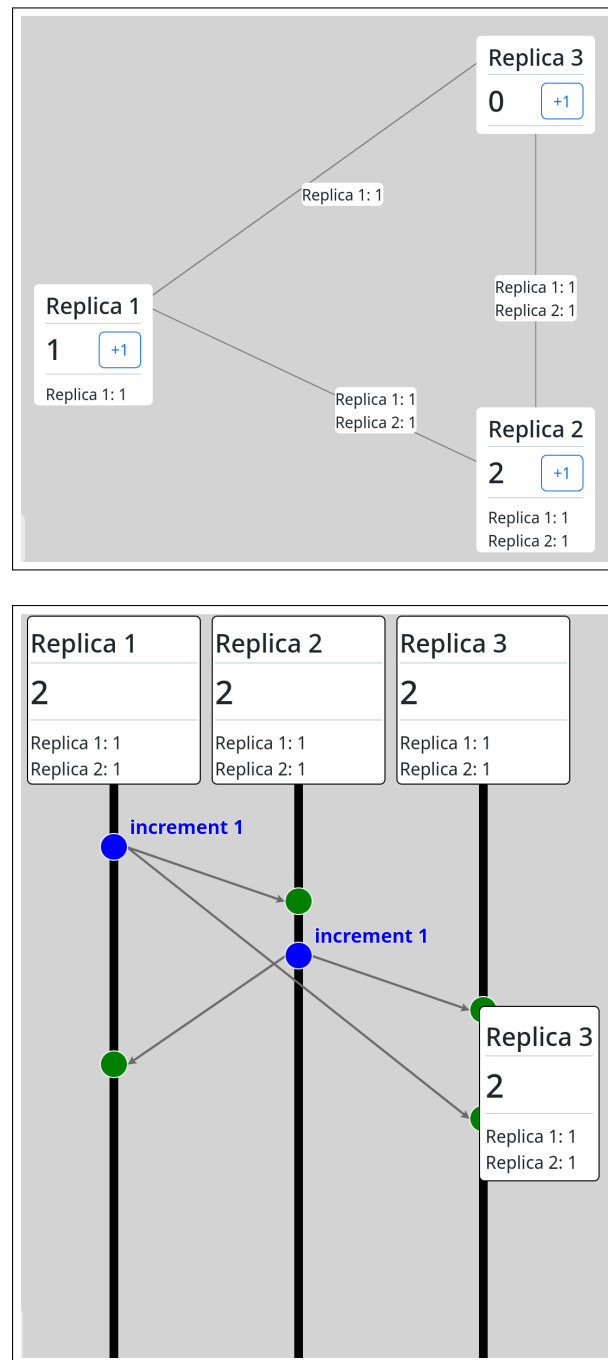


Figure 4.7.: The user sees both a network graph and a space-time diagram. In this example, the three messages in transit visible in the network graph have already arrived in the space-time diagram.

4.3. Design and Implementation of Visualizations

The *CRDT Showcase* by Philipp Lersch [21] is chosen as base for the implementation for a variety of reasons. Since the *CRDT Showcase* is a web application written in Typescript, platform compatibility is guaranteed so long as the user has access to a browser. As the system needs to handle a variety of data types, TypeScript's type system provides a major advantage over plain JavaScript. In addition to that, the ability to send messages between nodes in an implicitly fully-meshed network graph, as well as multiple CRDT-implementations, are already provided.

4.3.1. Architectural Changes

The *CRDT Showcase* was not designed with the addition of more visualization styles in mind. As a consequence, the classes primarily responsible for the frontend (*Showcase*) have high coupling with various classes in the backend (*Hub*, *Network*, *Node*, *CRDT implementations*). Many components integral to the architecture were written with a network graph visualization in mind. The class *TimedParcel*, for example, is responsible for animating messages during transit. When an animation has terminated, *TimedParcel* notifies the *Hub* to update the receiver-node which accesses the GUI in the *Showcase* class.

Some data structures stored their data as HTML elements in the frontend and fetched it via its HTML-*id* on every operation. The deletion of nodes was handled in the frontend and directly accessed data structures in the backend, while the creation of nodes was handled by the backend. In a small project with emphasis on development speed, this approach does not cause major issues, but technical debt accumulates with the addition of more features.

Therefore, the first development step was the separation of frontend and backend. All information concerning the state of a *Node* had to be stored in the *payload* of a *Node* object, without permitting storage of data in the *Showcase* class. The *Strategy* abstract class was significantly extended (see Section 4.5) and given the ability to output a node containing a name, state, and its operations as a single HTML string via the function *renderNode*. The majority of the system's backend logic was removed from the class *Showcase*, which is responsible for the majority of the frontend, and transferred to the *Hub* class. Early implementations of showcases, previously defined in the frontend, were removed from the frontend and added to the scripting component.

Despite these changes, the architecture is still purpose-built for a network graph view. The class *TimedParcel* is specifically tied to the sending and arrival of messages, including their probability of dropping or duplicating, to a visual animation. To work around this architecture, a *ViewPublisher* class was created. Whenever an important event occurs (such as a send event, receive event, node creation, node deletion *etc.*), the *ViewPublisher* is notified of this event. The *ViewPublisher*, in turn, publishes these events to other visualization styles. This Observer pattern, shown in Figure 4.9, enables the implementation of one or more alternative visualizations without making further alterations to the backend.

4.3.2. Implementation of Space-Time View

The space-time view is handled by a single class *SpaceTimeView*. In contrast to the Network graph, the space-time view displays its history and is therefore required to maintain a log of all events. Send and receive events are stored as objects of type *broadcastSender* or *broadcastReceiver*, respectively, and mapped to a number in the order they are registered by the *ViewPublisher*. A send event and all corresponding receive events are assigned a unique *broadcast-ID* and stored in a *broadcast-event*. This enables the system to identify all events belonging to a single broadcast, which is of importance for color-coding events. A simplified version of the relevant data structures is shown in 4.1.

Listing 4.1: Data structures used for logging in the space-time view.

```
events: Map<number, (broadcastSender |
                    broadcastReceiver)>

broadcasts: Map<broadcastID: string,
               broadcastEvent>

broadcastEvent {
    broadcastID: string;
    sender: broadcastSender;
    receivers: broadcastReceiver[ ];
}

broadcastSender {
    broadcastID: string;
    node: Node;
}

broadcastReceiver {
    broadcastID: string;
    node: Node;
}
```

The space-time view is depicted in Figure 4.8. At the top of each timeline, the CRDT’s node visualization function creates a representation of the node’s state. Underneath, events are ordered lowest to highest by log-entry. send events are blue, successful receive events are green, duplicated messages are yellow, and dropped messages are represented by red marks in the location they should have arrived at. Successful and duplicated messages connect their corresponding send and receive events with a gray arrow, while dropped messages are connected to their send events by a dashed red arrow. Arrows are drawn using the *LeaderLine* JavaScript library [4].

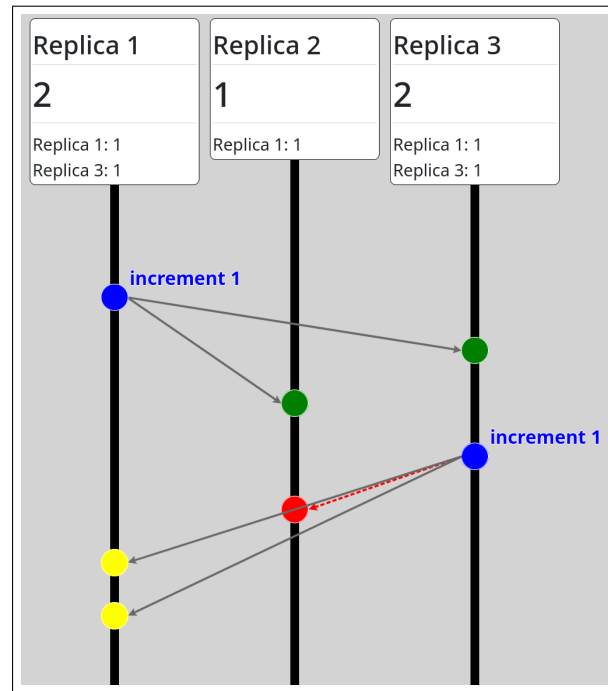


Figure 4.8.: Successful, dropped, and duplicated messages. Operations not yet included in Nodes.

4.3.3. Implementation of Meshing

Whenever an operation relevant to the network graph's meshing is executed, a notification is sent to the *viewPublisher*. Relevant operations include the creation of nodes, deletion of nodes, creation of new edges, and removal of existing edges. The *ViewPublisher* notifies the class *NodeMesh*, which draws a line between any two nodes in the *Showcase* for which a link is stored by the system. The resulting network graph is depicted in Figure 4.7. A high level overview of this architecture can be found in Figure 4.9. Edges are drawn using the *LeaderLine* JavaScript library [4].

4.4. Design and Implementation of Scripting Component

4.4.1. Script Structure

All operations available to the user are listed in Section 4.4.4. The operations available in the scripting component are a superset of all operations available via the GUI. A user may want to observe a CRDTs behavior in specific scenarios, including full network partitions, or networks where specific messages are dropped and duplicated. To facilitate this, the scripting component supplies the user with operations that guarantee successful, dropped, and duplicated broadcasts, as well as node-to-node messages, in addition to the simple ability

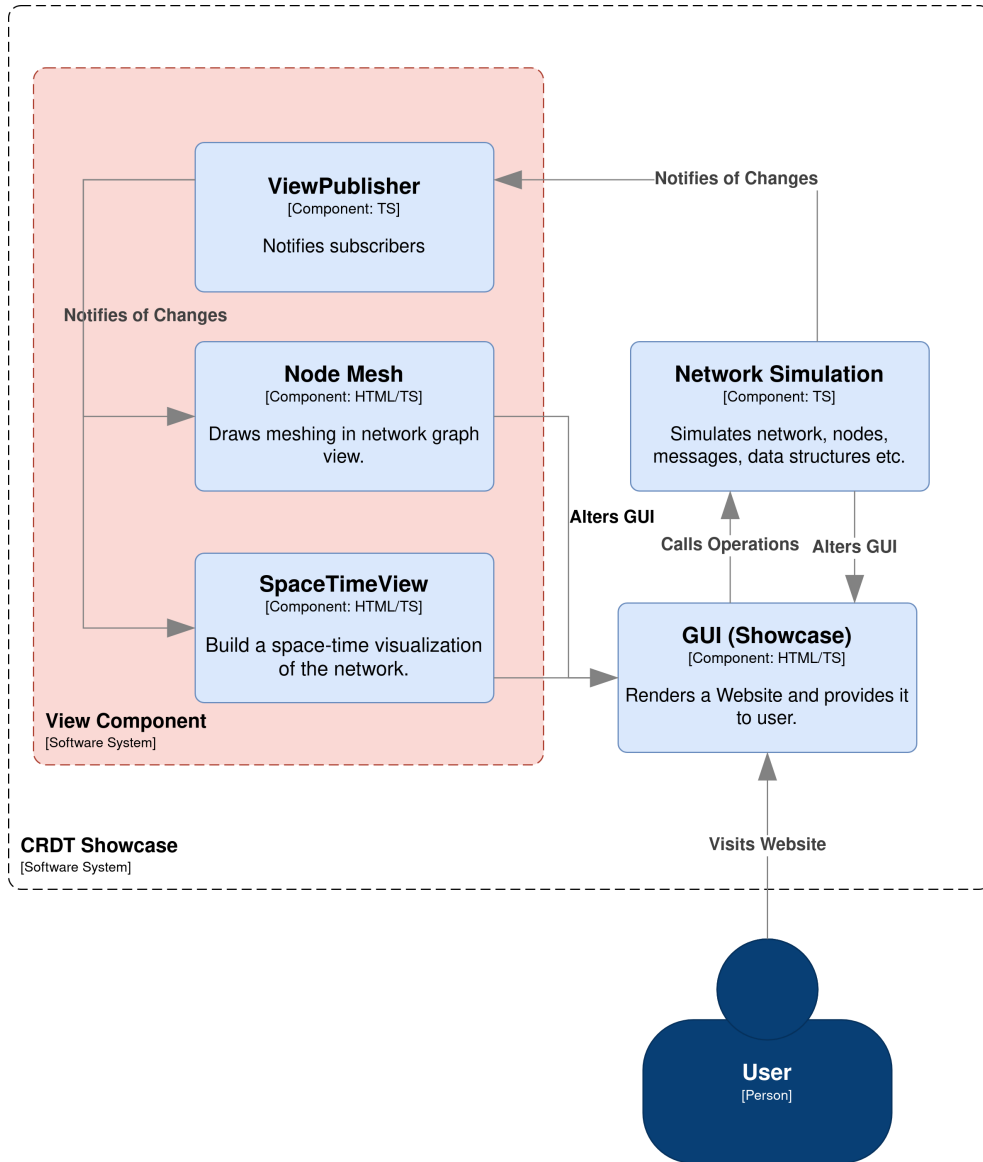


Figure 4.9.: A C4 diagram of the architecture with emphasis on the newly created **View** component. Most program logic is handled in the **Network Simulation** component.

to perform an operation whose success is dependent on current network settings. In addition, the user may decide to create a popup and alter the text displayed in it.

The scripting language was implemented with JSON Syntax. An exemplary function call

```
function(arg1, arg2, arg3)
```

with three parameters is scripted as a list in which the function name is the first element, encoded as string, while arguments are encoded as the second, third, *etc.* list elements of the type *string*:

```
["function", "arg1", "arg2", "arg3"]
```

A script is a single list in JSON syntax, which is, in turn, made up of nested lists. Therefore, the script must begin and end with a single opening and closing square bracket, respectively. Contained within this list are *function calls* and *procedure definitions*. All function calls are immediately executed in a line-by-line fashion when the user loads the script. They are, therefore, well-suited to setting up the system initially, including the creation of a network, data structure, and popup. A *procedure definitions* is a set of operations that is performed when the user clicks the *continue*-button of the showcase's popup.

A *procedure definition* consists of multiple function calls nested within a list:

```
[  
    ["function1", "arg1", "arg2"],  
    ["function2", "arg1", "arg2", "arg3"],  
    ["function3", "arg1"]  
]
```

Procedure definitions, like function calls, are registered top-to-bottom, meaning that the first defined procedure definition is executed first and the last defined procedure definition is executed last. To execute procedures, a popup must first be created in the initial function calls. The first procedure is executed when the user clicks the *continue*-button on the popup. From then on, clicking the *continue*-button on the popup will end the current procedure and start the next procedure.

A sample script may look as follows:

Listing 4.2: A sample script in JSON syntax. *columns*

```
0  [  
1      ["dataStructure", "S_pn_counter"],  
2      ["buildPopup"],  
3      ["THIS IS A COMMENT"],  
4      ["createNode", "node1"],  
5      ["createNode", "node2"],  
6      ["createNode", "node3"],  
7      ["changePopupText", "test1"],  
8      [  

```

```
9      ["THIS IS A PROCEDURE"]
10      ["popupTimeout", "3000"],
11      ["simulateOperationDropped", "node1", "
        increment", "1"],
12      ["changePopupText", "test2 "]
13  ],
14  [
15      ["popupTimeout", "3000"],
16      ["simulateOperationDuplicated", "node1", "
        increment", "1"],
17      ["changePopupText", "finished "]
18  ]
19 ]
```

When the above script is first loaded, the system first creates a network and selects a state-based PN-Counter as data type (l.1). Then, it builds a popup (l.2), creates three nodes (ll.4-6), and changes the popups content (l.7). Note that elements that do not conform to any defined operation, as shown in l.3, are ignored by the system and can be used as comments.

Once these initial operations have been performed, clicking the *continue*-button on the popup will trigger the execution of the first procedure (ll.8-14), in which the popup is hidden for three seconds (l.10), followed by the system incrementing the counter on *Node1* and dropping all update messages (ll.11-12). At last, the popup text is altered (l.13). Clicking *continue* on the popup will then trigger the execution of the second procedure definition (ll.15-20).

This syntax is, at first glance, unconventional, especially when compared to *C*-like syntaxes. Encoding function calls as function names and function arguments in a single list is not unheard of, though, and bears similarities to *s-expressions* in LISP-like languages, which may encode a function call as *(function arg1 arg2 arg3)* [35].

The syntax's verbosity may present an inconvenience to the user. All function calls and procedures definitions must be enclosed in square brackets and separated by comma. The user is required to enclose all operation names and arguments in quotation marks.

However, despite its verbosity compared to other viable syntaxes, JSON is one of the most-used data types in web applications. It has a simple, human-readable grammar [8] and, due to its high adoption, is very likely to be known to a large number of users.

A major advantage of a JSON-like syntax is its ease of parsing. The JavaScript command `JSON.parse` can, in a single line, create an object from the script. It is therefore not necessary to build a compiler from ground-up, as both scanning and parsing are handled by a single command. The programmer can immediately begin working with a syntax-tree, encoded as a nested list. On the top-level, the *PresetParser* only needs to match either a *string*, a *list*, or a *list of lists*. Since all list elements are encoded as strings, the *PresetParser* can simply match a given operation in a switch statement and catch any syntax

errors with a *default*-statement.

4.4.2. Showcases

A variety of showcases as described in Section 4.1 can be implemented to explain the properties of CRDTs. The following list introduces multiple options for possible showcases that have been implemented or can be implemented in the future.

- A showcase "**CRDT Introduction**" provides introductory information on CRDTs, including their definition as per Section 2.1. The showcase creates two unlinked nodes with a state-based CRDT and either performs operations on them or prompts the user to perform operations on them, resulting in divergent states. Then, the system connects the two nodes and performs operations on both of them, causing them to propagate their updates to each other. The user can see that both nodes have reached the same state after the messages' arrival.
- A showcase "**Operation vs. State-based CRDTs**" contrasts said types of CRDTs. A possible sequence of operations may look comparable to the test scenario in Section 5.2.2. This showcase is able to explain the difference in update propagation between the two CRDT types by creating two networks of the corresponding CRDT type and explaining each update propagation method via popup messages.
- A showcase "**Building a decrementable counter**" introduces the user to a state-based vector counter first, then explains how adding a second vector to the data structure enables decrements.
- In a similar fashion, a non-crdt set can be contrasted to a 2P-Set CRDT.
- The capabilities and advantages of individual CRDTs can be explained in individual showcases.

4.4.3. Scripting architecture

It must be noted that the use of the word *showcase* in the system bears risk of confusion. To disambiguate: The class *Showcase* is primarily responsible for the GUI. A single *Showcase*, as described in Section 4.1, is a set of automatically executed operations, complemented by textual explanations, selectable by the user. To avoid confusion with the *Showcase* class, most classes belonging to the scripting component contain the term *preset* instead of *showcase*.

The scripting component's implementation required minor architectural restructuring. Operations previously implemented by the *Showcase* were transferred into the *Hub*, from where the *Showcase* can call functions. Likewise,

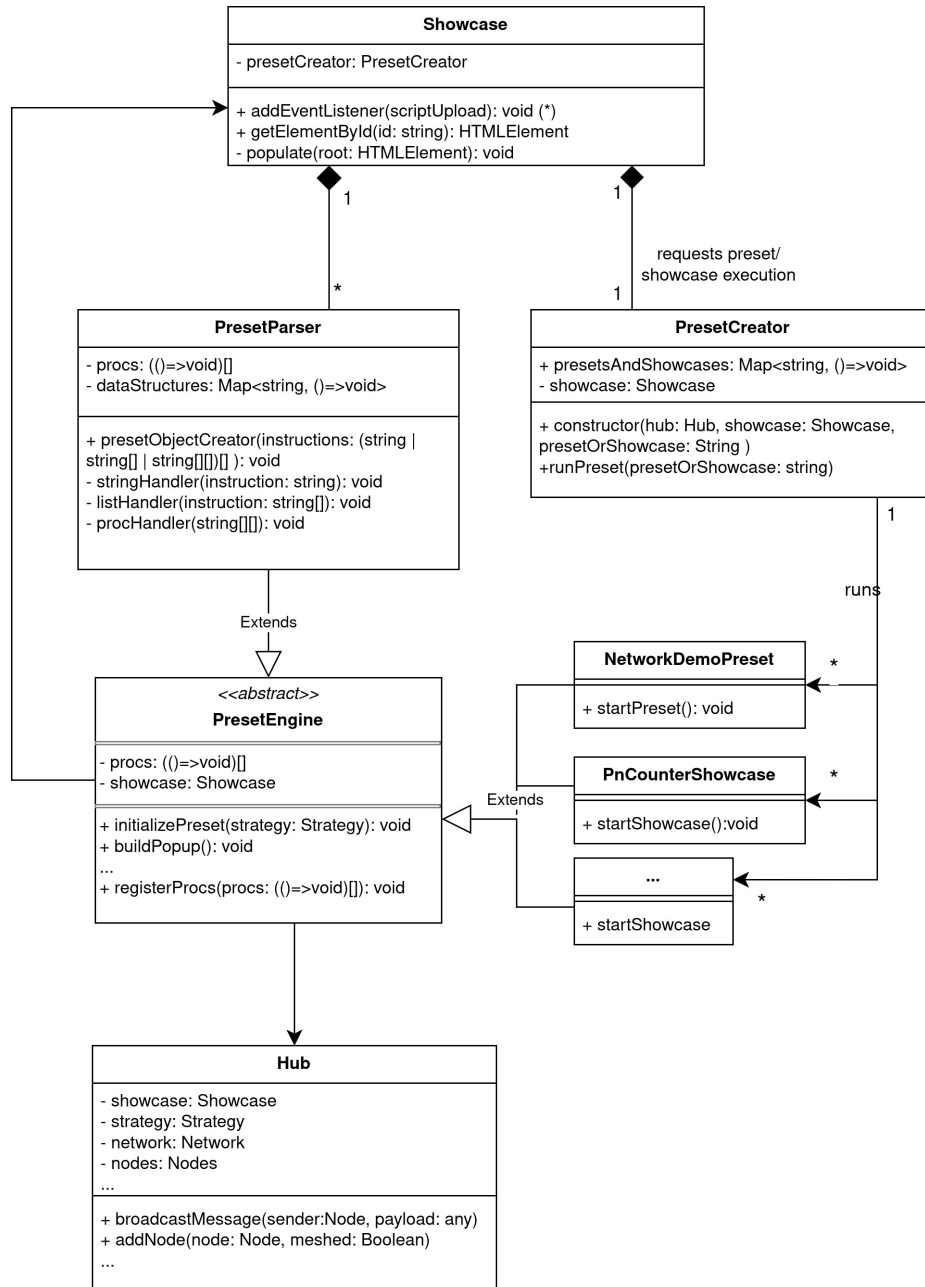


Figure 4.10.: A reduced UML diagram of the scripting architecture.

the *Strategy* class needed minor architectural changes. Previously, send operations were implemented as part of an event-listener attachment. This structure would have required the simulation of a click event in the *Showcase* by the scripting components, which creates unnecessary coupling of frontend and backend. Therefore, the logic required for an operation on the data type was moved into a separate function *doOperation*, which is detached from the event-listener registering operations triggered via the GUI (see Section 4.5.1).

The diagram in Figure 4.10 is a reduced class diagram of the scripting architecture. Only those classes directly implementing or highly relevant to the system's scripting capabilities are included, in full or abbreviated fashion.

The *presetEngine* implements the operations defined in Section 4.4.4 by calling functions in the *Hub*, and to a lesser degree in the *Showcase*. Procedure definitions are stored in an array of anonymous functions *procs: (()=>void)[]*, which is set by the *registerProcs* function.

If the user uploads a script, the *Showcase* parses the script, creates an object from it, creates a *PresetParser* and lastly calls the function *PresetParser.presetObjectCreator*, which takes the object created by the parsed script as argument. The *PresetParser* is capable of handling strings (operations without parameters), lists of strings (operations with and without parameters), and lists of lists of strings (procedure definitions). Strings and lists of strings from the parsed script are matched with operation names line by line and then executed by calling inherited functions from the *PresetEngine*. Procedure definitions are converted to anonymous functions and pushed to the *procs: (()=>void)[]* array, through which the user iterates whenever they click *continue* on a popup.

The classes *NetworkDemoPreset*, and *PnCounterShowcase* contain a *Preset* and a *Showcase*, respectively, while the class named *"..."* is representative of all other presets and showcases not included in the class diagram due to their large number. Each class is a subclass of *PresetEngine* and has a single function in which a preset or showcase is defined. Since instantiating a new object for every single preset and showcase in the *Showcase* class leads to a confusingly high amount of dependencies, the creation of preset and showcase objects is handled by the *presetCreator* class. The *presetCreator* class stores a key-value map in which the key is a preset or showcase name, while the value is an anonymous function that creates a preset or showcases object and immediately runs said preset/showcase. The *Showcase* class needs only a single *presetCreator* object to run all available presets and showcases.

4.4.4. Supported Operations

The below operations are supported by the *PresetEngine*. Some operations require data structures, operation types, or operation arguments to be passed as parameters. These data structures and their permitted operations and arguments are documented in Section 4.5.2.

initializePreset(*strategy*)String *strategy*Strategy *strategy*

Initializes the network with a given data structure *strategy*. Must be called before all other operations. Resets the network and data structure if they already exist. *strategy* specifies the type of data structure as a Strategy object in code and a *string* in user-defined scripts. A list of strategies is found in Section 4.5.2.

buildPopup()

Initializes an empty popup window. Must be called before content of popups can be altered.

destroyPopup()

Removes popup window from view.

changePopupText(*text*)String *text*

Makes the popup display *text*. Accepts plain text and HTML.

popupTimeout(*duration?*)Number *duration*

Hides popup for specified duration. The *duration* parameter is optional. If not specified, the popup will be hidden for (Network Latency + 1.5s). If specified, the value is interpreted in milliseconds.

createNode(*name*, *meshed*)String *name*Boolean *meshed*

Creates a new node with name *name*. If *meshed* is true, the new node will be meshed with every other node in the network. If false, the node is not connected to any other node.

toggleNodeOffline(*name*)String *name*

Changes status of node with name *name* from online to offline and vice versa, depending on its current status.

deleteNode(*name*)String *name*

Deletes node with name *name*, if it exists.

simulateOperationSuccess(*nodeName*, *op*, *arg*)String *nodeName*String *op*String *arg*

Performs an operation *op* on a node *nodeName* and then sends an update to all neighboring nodes. The update is guaranteed to be delivered exactly once to each node. *op* may take an argument *arg*, depending on the previously specified data structure.

simulateOperationDropped(*nodeName*, *op*, *arg*)

String *nodeName*

String *op*

String *arg*

Like *simulateOperationSuccess*, but all outgoing messages are guaranteed to be dropped.

simulateOperationDuplicated(*nodeName*, *op*, *arg*)

String *nodeName*

String *op*

String *arg*

Like *simulateOperationSuccess*, but all outgoing messages are guaranteed to be duplicated.

simulateOperation(*nodeName*, *op*, *arg*)

String *nodeName*

String *op*

String *arg*

Performs an operation *op* with argument *arg* on a node *nodeName* and then sends an update to all neighboring nodes. Whether the operation arrives successfully, drops, or is duplicated depends on the current network setting.

setDuplicationRate(*probability*)

Number *probability*

Sets probability of outgoing messages being duplicated. *probability* must be between 0 and 100.

setDropRate(*probability*)

Number *probability*

Sets probability of outgoing messages being dropped. *probability* must be between 0 and 100.

setLatency(*latency*, *range*)

Number *latency*

Number *range*

Sets network's latency and latency range, measured in milliseconds. *latency* sets the minimum travelling time of a message. A random value between 0 and *range* is added on top of the *latency*.

createFullMesh()

Creates a connection between every single node of the network.

removeMesh()

Removes every connection between nodes in the network.

addLink(*node1*, *node2*)

String *node1*

String *node2*

Creates a direct connection between *node1* and *node2*

removeLink(*node1*, *node2*)

String *node1*

String *node2*

Removes a connection between *node1* and *node2*

nodeToNodeMessage(*sender*, *receiver*, *op*, *arg*)

String *sender*

String *receiver*

String *op*

String *arg*

Executes operation *op* with argument *arg* on node *sender* and sends a message from *sender* to *receiver*, ignoring the networks meshing.

4.5. Design and Implementation of Data Structures and Node Visualizations

Data Structures are implemented as extensions of an abstract class *Strategy*, the operations of which are detailed in Section 4.5.1. The *Strategy* class has access to a *Node* class's payload, which contains state-information, and can trigger broadcasts via the *Hub* class. A reduced class diagram of this architecture is found in Figure 4.11.

Visual representations of nodes are composed of three independent parts:

- (i) The top component contains the node's name.
- (ii) In the middle component, the system displays the data type's available operations. For some counter-based data types, the counter is displayed in this section to save space.
- (iii) In the bottom component, the data structure's current state is displayed.

An example for a node with a PN-Counter CRDT is shown in Figure 4.12. The middle section displays the data structure's available operations (an increment and a decrement), as well as the cumulative counter calculated from

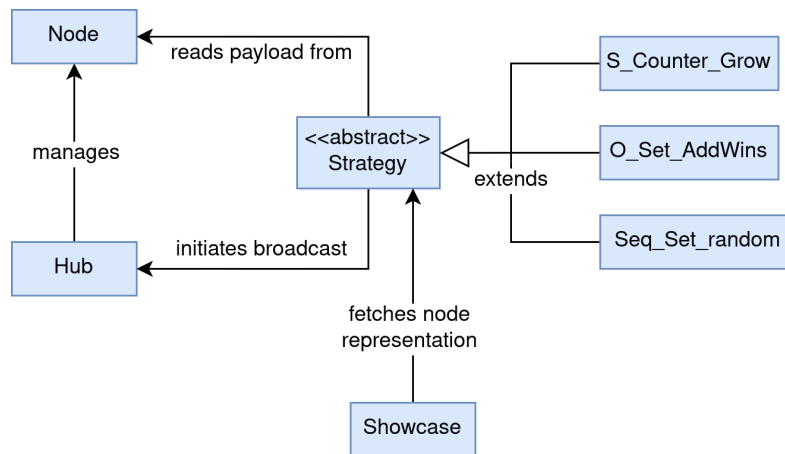


Figure 4.11.: A reduced class diagram focussed on the *Strategy* class and its interaction with other system components.

the node's state. The bottom section displays the node's current state, excluding the cumulative counter: A vector (here implemented as a Map) storing a positive and a negative value for each known node.



Figure 4.12.: A node containing a PN-Counter.

As described in Section 4.3.1, the system's original architecture was not built with multiple views in mind, which resulted in high coupling between classes when constructing a node: the basic HTML skeleton of a node was defined in the *Node* class, which called various functions from its *Strategy* implementation. Some alterations to the node's state were implemented by fetches from and direct calls to specific components in the *Showcase*.

While this approach is feasible when working with a single view, the addition of one or more views creates the need for a mechanism capable of displaying the same node in multiple views. For this purpose, the *Strategy* abstract classes was appended with a *renderNode(node: Node)* function, as described in Section 4.3.1. As a result, each data structure is required to be capable of rendering a complete node (see Figure 4.12) as HTML string. The specific functions implementing a node's visualization are documented in Section 4.5.1. When a

view updates a node as a consequence of an operation or a receive event, it is able to call *Strategy.renderNode* and replace the corresponding node's HTML representation without altering the node's state.

The resulting architecture enables a simple implementation process for new CRDTs, since the implementation is not reliant on understanding the entire website's HTML structure, thereby conforming to the maintainability requirements in Section 2. The data structure is defined in the new CRDT's class, the node's HTML representation is created by the CRDT implementation without regard for the rest of the GUI, and the corresponding event listener for the node's buttons is attached to the *id* assigned to the buttons in the HTML representation.

A large variety of data structures are available to the user. Some of these data structures were already present in the original *CRDT Showcase* and merely rewritten to conform to the new architecture, others were added during the implementation process. A list of all data structures and their available operations is found in 4.5.2.

4.5.1. The Strategy Abstract Class

The below documentation shows the *Strategy* class's most important properties and functions. Functions denoted with a star (*) were added as part of the architectural restructuring.

name: string
The data structure's name.

description: string
The data structure's description to be displayed in the *Showcase*

type: ("State based" | "Operation based" | "Assumed sequential")
The node's data structure type.

***displayName**(*node: Node*): string
Renders the node's top content by retrieving its name and embedding it in an HTML string. In the majority of cases, overwriting this function is not necessary.

***displayOperations**(*node: Node*): string
Renders the node's middle content. Must be implemented for every data type.

***displayState**(*node: Node*): string
Renders the node's bottom content. Must be implemented for every data type.

***renderNode**(*node: Node*): string

Embeds *displayName*, *displayOperations*, and *displayState* in an HTML template and returns it as a string. Typically, overwriting this function is not necessary, as the data types operations and state are rendered by *displayOperations* and *displayState*.

initUpdateContent(*payload: any*): string

Renders the content of a live message. Must be implemented for every data type.

***doOperation**(*node: Node*, *hub: Hub*, *op: string*, *arg?: string*): void

Performs an operation *op* with an optional argument *arg* on the node, then broadcasts a message to all neighboring nodes by calling *hub.broadcastPackage*. Must be implemented for every data type.

registerEventHandler(*node: Node*, *hub: Hub*): void

Attaches an event handler to the node's operations in the middle section. Calls *doOperation*

update(*sender: Node*, *receiver: Node*, *payload: any*, *hub: Hub*): void

Updates the *receiver*-node's state when receiving a message containing a *payload*.

4.5.2. CRDT types

Listed below are data structures supported by the system, including their names, operations and arguments as expected in a *script*. Many of these data structures were already present in the original *CRDT Showcase*.

- **PN-Counter(State-based):** '*S_pn_counter*'

op	arg
"increment"	Any <i>arg</i> of type <i>String</i> will be ignored.
"decrement"	Is always "1".

- **Grow-only vector counter(State-based):** *'S_vector_counter'*
- **Non-idempotent counter(Op-based):** *'O_no_idem_counter'*
- **Sequential Counter:** *'Seq_random_counter'*

op	arg
"increment"	Any <i>arg</i> of type <i>String</i> will be ignored. Is always "1".

- **2 Phase Set (State-based:)** *'S_2p_set'*
- **Add-wins set (Op-based):** *'O_add_wins_set'*
- **Simple set:** *'Seq_random_set'*

op	arg
"add"	Arbitrary String
"remove"	

- **Last-write-wins Register(Op-based):** *'O_lww_register'*

op	arg
"write"	Arbitrary <i>String</i> .

5. Evaluation and Discussion

5.1. Architecture Evaluation

The architecture was envisioned with maintainability as defined in Section 2 in mind. The majority of architectural decisions and trade-offs are discussed in the individual sections of chapter 4 as part of the system’s design process. In this section, the system’s adherence to the *modularity* quality measure is evaluated by counting how many classes a user has to modify to which degree in order to create a new component.

This architectural change of the view component significantly improves modularity and modifiability as defined in Section 2. The introduction of a new view requires changes in three classes:

1. In the *Showcase* class, responsible for the GUI, a dedicated pane and a button used to switch to the new view have to be introduced. This is a minor change.
2. The new view’s class must be added to the *ViewPublisher*. This is a minor change.
3. The new view is implemented as a class responsible for all tasks related to the new view. Only the newly introduced pane in the GUI is altered by the new class.

Evidently, a new view can be introduced without extensive modification of pre-existing code. A new view operates on its own pane, implemented as a *div* element, and therefore does not interfere with other views.

The introduction of a new showcase requires changes in two classes:

1. A new class extending the *PresetEngine* class must be created. This class contains the new showcase and can be implemented to exclusively use operations defined in the *PresetEngine*.
2. The new class containing the showcase must be manually registered in the *PresetCreator*. This is a single entry in a *Map*.

After these two steps, the GUI automatically registers the new showcase from the *PresetCreator* and adds it as a selectable option. This architecture enables users to create new showcases in the source code with minimal alterations to the rest of the system. The user is not required to make any calls to the GUI or the network simulation whatsoever. Therefore, the maintainability requirement in Section 2 is fulfilled to a high degree.

The addition of a new CRDT requires changes in two classes:

1. A new class extending the *Strategy* abstract class must be created. This class implements the CRDT.
2. The new class must be registered in the global data structure *strategiesStructured*, which is then accessed by the GUI.

While the creation of a new CRDT requires only the creation of a new class and a single entry in a global data structure, the CRDT class may have high coupling with other classes. State data is stored in *Node* classes and calls to the network simulation are made frequently. Furthermore, despite the architectural restructuring, CRDT implementations may need to access the GUI directly, depending on how their event listeners are implemented. Nonetheless, no classes besides the new CRDT's class and a single global data structure must be altered, which satisfies the modularity requirement, though the high coupling with other classes has the potential to violate the modifiability requirement as per Section 2.

In conclusion, the system is modular and modifiable enough to satisfy the maintainability quality measure to a sufficient degree.

5.2. Qualitative Performance Evaluation

In order to evaluate the system's functional suitability, a set of scenarios users might wish to explore is formulated and then solved with the system's capabilities.

5.2.1. Complete Network separation

A user wishes to observe the behavior of state-based CRDTs if operations are applied in a bisected network. After some time, the user wishes to observe how the CRDT deals with reconnection of both network components and whether all updates performed during the bisection period can be propagated to all nodes.

To solve this task, the user writes Script A.1, which creates a bisected network, consisting of two components containing two nodes each. The user creates procedures which execute different operations in each of the two network components so that nodes from different components have strongly diverged in state. The result is shown in Figure A.1. The network separation is easily distinguishable in both visualization types, as is the state convergence. In one network component, the nodes' counter displays "2", while the nodes in the other network component display "-2".

Next, the user's script reconnects all nodes to create a fully meshed network and sends two update messages—one from each former network component, respectively. The result is Figure A.2. All nodes visibly have the same state. In the space-time diagram, we see that one message from each previous component

has reached all nodes from the other previous component, updating their states. Therefore, the user has the ability to create a script showcasing strong eventual consistency of state-based CRDTs in a bisected network.

It is noted that this scenario can strongly benefit from a state graph as described in Section 4.2.3 to showcase strong eventual consistency. In a state graph, both network components would visibly diverge during the initial operations and then converge to the same final state.

5.2.2. Operation vs. State-based CRDTs in Unreliable Networks

A user wishes to compare state-based and operation-based CRDTs in unreliable networks via the GUI. Specifically, the user wants to contrast their behavior when they are faced with package loss.

In the GUI, the user selects the state-based 2P-Set and creates three nodes via the menu's "Replicas" section. In the GUI's "Network" section, the user slides the "Drop Rate" slider to "100%" and the "Duplication Rate" slider to "0%". Then, the user adds two elements "a" and "b" to Replica 0. The network drops both update messages, causing state divergence between nodes.

Afterwards, the user toggles the "Drop Messages" button in the GUI's "Network" section and adds an element "c" to Replica 0. A message containing the state {a,b,c} travels to all other nodes in the network graph view and updates their state.

The result of these operations is shown in Figure A.3. All nodes visibly have the same state after the last operation. In the space-time diagram, we see that Replica 2's state was empty after the addition of "b" in Replica 0. Despite the loss of four packages, visible in the space-time diagram, the last two successful messages brought all nodes to the same state on arrival.

Next, the user performs the exact same set of operations on an operation-based Set-CRDT. For this, they choose the Add-Wins-Set operation-based CRDT. The result in Figure A.4 shows that Replica 1 and Replica 2 diverge from Replica 0 even after the last update was successfully delivered. Since Replica 0 only ever propagates its operation instead of its state, the replicas' states irrecoverably diverge without an underlying reliable broadcast algorithm guaranteeing delivery.

After performing the exact same set of operations on a state-based and an operation-based CRDT, the user concludes that state-based CRDTs are capable of recovering from package-loss if they can propagate updates at a later point, while operation-based CRDTs require an underlying reliable broadcast to prevent state divergence.

5.2.3. CRDT Introduction

A user has heard of CRDTs in passing and is aware they are replicated data structures in distributed systems. However, text-based descriptions of CRDTs have proven themselves too abstract for intuitive understanding. To rectify this, the user wishes to view a real

CRDT in action, with additional textual descriptions describing basic principles of CRDTs.

The user navigates to the GUI's "Showcase" section and selects the "Introduction to CRDTs" option. This showcase provides a basic introduction to CRDTs, giving the definition and basic explanations from Section 2.1 in the popup. The basic properties of CRDTs are explained by first prompting the user to execute operations on two offline nodes, resulting in divergent states, then toggling both nodes online, resulting in propagation of their own state and eventual state convergence of both nodes.

5.3. Visualization Style Evaluation

Most advantages and disadvantages of implemented visualization styles are already considered in Section 4.2. The system was constructed to fulfill every requirement in Section 4.1.

In this section, the space-time view's color coding is of major interest. Events in the space-time view are colored by the event type they represent: send, successful receive, duplicated receive, and dropped message. This color-coding is independent of the individual node any event occurs on. The same type of event has the same color on any timeline. While this visualization style provides a detailed overview of a single operation's effects, it may cause confusion in concurrent operations, when events need to be disambiguated by node of origin.

An alternative approach is shown in Figure 4.4, where all nodes are assigned an individual color which is used to mark the node's send events, the subsequent receive events, and outgoing arrows. This approach may be preferable in networks with many nodes and concurrent operations and can, due to the system's modular architecture, be added as an additional visualization style, allowing the user to switch between two different space-time diagrams with different visualization schemes.

Additionally, as is elaborated in Section 5.2.1, the system may benefit from a state graph view to showcase eventual state convergence.

5.4. Quantitative Performance Evaluation

The system's performance was evaluated by visually observing stutter during live message passing in the network graph. Stutter was measured against (i) the number of nodes in the network, and (ii) the number of simultaneous send events. If there were fewer nodes than send events, individual nodes were instructed to send more than one message. A score between 0 and 2 was assigned depending on the severity of stutter:

	1	2	3	4	5	6	7
1	0	0	0	0	0	0	0
2	0	0	0	0	0	0	0
3	0	0	0	0	0	0	0
4	0	0	0	0	0	0	0
5	0	0	0	1	1	1	1
6	0	0	1	1	1	1	1
7	1	1	2	2	2	2	2

Table 5.1.: *Stutter severity test. Rows: Number of nodes. Columns: Global number of send events.*

- 0: No visible stutter.
- 1: Visible stutter, but the user does not lose track of individual messages even if they are near other messages.
- 2: Stutter severe enough that the user may temporarily lose track of individual messages if they are near other messages.

The evaluation was conducted on a medium-powered computer representative of most student's hardware (for specifications, see table A.1). The results are shown in table 5.1. Noticeably, there is no stutter in a network of four or fewer nodes, even if seven send events are executed simultaneously, resulting in 21 messages. In network sizes of five to six nodes, stutter is non-existent to mild, even in the most resource-intensive case of 35 simultaneous messages. Severe stutter becomes a problem in systems with more than seven nodes and three simultaneous send events, resulting in at least 18 simultaneous messages.

Since the system is designed as an educational tool, and not a simulation engine for large distributed systems, stutter in large networks is deemed acceptable. No showcase defined in the system and no user-created script envisioned in this thesis (see Scripts A.1 and 4.4.1) exceeds any threshold that may cause stutter.

5.5. Limitations

5.5.1. Node Size

The complexity and size of nodes' states may be inherently limited by their size. As observed in Figure 5.1, a PN-Counter in a network of just six nodes may result in large visualizations of the nodes' states, cluttering the view with overly large nodes. The same phenomenon can occur if many elements are added to a set-based CRDT.

This issue can be ameliorated in the individual implementations of CRDTs by manually altering the function responsible for rendering the node's bottom

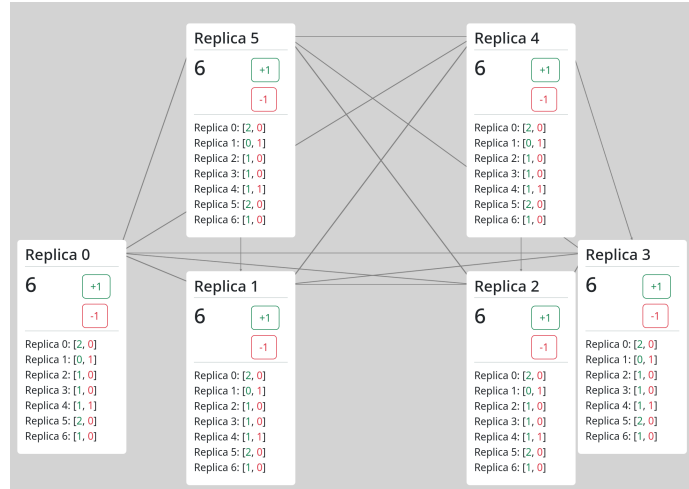


Figure 5.1.: A system of just six nodes may result in impractically large states.

section containing the CRDT. If a state becomes too large, programmers can split it into a visible and a hidden element, where the hidden element becomes visible if the user hovers over or clicks the visible element. In the PN-Counter, for example, programmers can limit the visible vector size to four entries and display any more entries if the user hovers over the visible state. This approach gives programmers a large amount of control over the nodes' representations, but bears the risk of cluttering the visualization when working with unoptimized implementations of state representations.

5.5.2. Viability of Large Networks

Due to the possible node size and performance limitations explained in sections 5.5.1 and 5.4, the system is not capable of handling large networks. The exact limitations are dependent on CRDT implementation, screen size, number of concurrent operations, and computation power, but a tentative recommendation to limit the network's size to a maximum of ten nodes, ideally fewer, can be made.

This limitation, however, is not expected to cause issues for users. None of the showcases and test scenarios in sections 4.4.2 and 5.2 require a network size of more than six nodes. The algorithms and data structures the system seeks to visualize can be explained without exceeding the recommended number of nodes.

5.6. Future Work

The system's modularity permits for many functionalities to be added, the most obvious being additional views, data types, and showcases. Additional views are covered in Section 5.3, while possible showcases are covered in Section 4.4.2.

Of particular interest is the addition of new data structures. Currently, only different varieties of sets, counters, and registers are implemented. More complex CRDTs using list or tree semantics, as mentioned in Section 2.2.4, are not implemented. The visualization of lists or trees requires the implementation of a new state visualization scheme. Since a visualized tree or list requires more space than a visualized set or counter data structure, a view-on-hover functionality as described in Section 5.5.1 would become a necessity.

In order to prevent large state visualizations from cluttering the views, a functionality to automatically hide states exceeding a predefined height or width limit may be considered, potentially simplifying the implementation of complex data structures.

While this work is focussed on CRDTs, many other classes of algorithms exist in the domain of distributed systems, a major example being broadcast algorithms. It is questionable whether a broadcast algorithm can be visualized with the system's current capabilities, considering that the system simulation's *broadcast* function, called by a node, simply sends a message to every single neighboring node. In order to implement different types of broadcast algorithms, like reliable or best-effort broadcast, a *BroadcastStrategy* abstract class, from which multiple broadcast implementations inherit their functions, can be implemented as part of a strategy design pattern. If a broadcast algorithm requires state information to be stored by nodes, an additional property *broadcastData* may be introduced to the *Node* class. Optionally, this broadcast data store may add an additional component to the nodes' visual representation.

6. Conclusion

The purpose of this thesis was the visualization of distributed algorithms with a focus on Conflict-Free Replicated Data Types (CRDTs), showcasing their behaviors, advantages, and disadvantages in an interactive and comprehensible manner. In addition, a pre-existing tool was modified and extended to facilitate the exploration of these concepts by allowing users to create and visualize their own network scenarios.

Throughout this thesis, various visualization styles for CRDTs were compared, further developed, and added to the system in order to satisfy a variety of visualization requirements. Executing operations on CRDTs within these visualization styles allows for a more intuitive understanding of their behaviors. Moreover, interactive showcases explaining different CRDT behaviors were integrated, providing an engaging way for users to learn about the inner workings of these data structures. A scripting grammar was also defined, enabling users to upload custom network simulations into the system.

Architectural restructuring ensured that the system remained modular, allowing for the easy addition of new CRDT implementations, visualization styles, and scripting functionality. Additional components can be added with few modifications to existing components, making the system adaptable to future extensions.

The results of this thesis demonstrate that the system successfully addresses several use cases, providing a solid framework for visualizing and interacting with CRDTs in distributed networks. The tool works effectively for smaller, individual network scenarios, as intended. While the system may not handle large-scale simulations with numerous nodes, this limitation is acceptable given that the tool is primarily designed to focus on the behavior of a few key nodes.

In conclusion, this thesis has contributed to the understanding and visualization of CRDTs and distributed algorithms, providing an accessible and interactive tool for developers and students interested in exploring these concepts. The tool's flexibility and extensibility open up many opportunities for future enhancements.

List of Figures

3.1.	A space-time diagram from Pregoça <i>et al.</i> [33] used to showcase an Observed-Remove Set.	12
3.2.	ShiViz [5] places the space-time diagram next the logs it was reconstructed from.	13
3.3.	The Raft visualization from [25] passes live messages and displays the system’s current global state.	13
3.4.	This visualization mixes text-based explanations with live message passing, but only shows two nodes [20].	14
3.5.	This instance of the <i>CRDT Showcase</i> is configured to show three nodes with a simple counter data structure. <i>Replica 3</i> has previously incremented its own counter, resulting in two messages with the content (1) travelling to <i>Replica 1</i> and <i>Replica 2</i> . Options and settings are located in the top left corner.	15
3.6.	A tree diagram visualizing Treedoc’s data structure [29].	18
3.7.	An excerpt from a graphic in [30] visualizes a part of the CRDTs underlying data structure: a linked list.	18
4.1.	A large number of messages in a fully-meshed network containing as few as six nodes cause significant clutter.	23
4.2.	A space-time diagram visualizing a system’s history and its current state. A set-based CRDT is used. ¹ A node’s current state is displayed above the timeline. Users can create new set elements via the add:_ field. ² If the user hovers over an event, the node’s state at the time of the event is displayed. ³ Messages display their content during transit. Here, the messages contains Node1’s state after the operation add(b)	24
4.3.	A space-time diagram visualizing meshing. Edges between nodes are colored blue, while the horizontal gray below the nodes separates the nodes from the timelines.	24
4.4.	Despite the diagram’s color-coding, a large number of events and messages can make a space-time diagram confusing to the user.	25
4.5.		27
4.6.	A state graph in which transitions of each node are marked in a unique color.	27
4.7.	The user sees both a network graph and a space-time diagram. In this example, the three messages in transit visible in the network graph have already arrived in the space-time diagram.	29
4.8.	Successful, dropped, and duplicated messages. Operations not yet included in Nodes.	32

4.9. A C4 diagram of the architecture with emphasis on the newly created View component. Most program logic is handled in the Network Simulation component.	33
4.10. A reduced UML diagram of the scripting architecture.	37
4.11. A reduced class diagram focussed on the Strategy class and its interaction with other system components.	42
4.12. A node containing a PN-Counter.	42
5.1. A system of just six nodes may result in impractically large states.	52
A.1. A bisected network in which both components have strongly diverged from each other.	61
A.2. The bisected network graph has been reconnected and all nodes have registered all updates executed during the bisection period. All nodes have the same state.	62
A.3. A state-based CRDT can propagate past updates despite frequent package loss.	63
A.4. An operation-based CRDT cannot propagate past updates in a network prone to package loss without an underlying broadcast algorithm.	64

A. Appendix

Listing A.1: *A user-created script to showcase recovery after complete network separation. columns*

```
0  [
1    ["dataStructure", "S_pn_counter"],
2    ["buildPopup"],
3    ["createNode", "node1"],
4    ["createNode", "node2"],
5    ["createNode", "node3"],
6    ["createNode", "node4"],
7    ["removeMesh"],
8    ["addLink", "node1", "node2"],
9    ["addLink", "node3", "node4"],
10   ["changePopupText", "This network is split into
      two disconnected components. Let's perform
      some increment operations on one part of the
      network:"],
11   [
12     ["simulateoperationSuccess", "node1", "
        increment", "1"],
13     ["simulateOperationsuccess", "node2", "
        increment", "1"],
14     ["popupTimeout", "4000"],
15     ["changePopupText", "Now, let's perform some
        decrement operations on the other part of
        the network:"]
16   ],
17   [
18     ["simulateOperationSuccess", "node3", "
        decrement", "1"],
19     ["simulateOperationSuccess", "node4", "
        decrement", "1"],
20     ["popupTimeout", "4000"],
21     ["changePopupText", "Now, let's see what
        happens if we reconnect the entire network
        apply some updates:"]
22   ],
23   [
24     ["addLink", "node1", "node3"],
25     ["addLink", "node1", "node4"],
```

```

26      ["addLink", "node2", "node3"],
27      ["addLink", "node2", "node4"],
28      ["simulateOperationSuccess", "node3", "
        decrement", "1"],
29      ["simulateOperationSuccess", "node1", "
        increment", "1"],
30      ["popupTimeout", "4000"],
31      ["changePopupText", "All updates have been
        applied to each node. All nodes have the
        same state."]
32    ]
33
34 ]

```

Model	Microsoft Surface Pro 8
CPU	11th Gen Intel Core i5-1135G7, 8 cores
GPU	Intel Iris Xe Graphics, 1.30 GHz (Integrated)
Memory	8 GB

Table A.1.: *Stutter test machine specifications.*

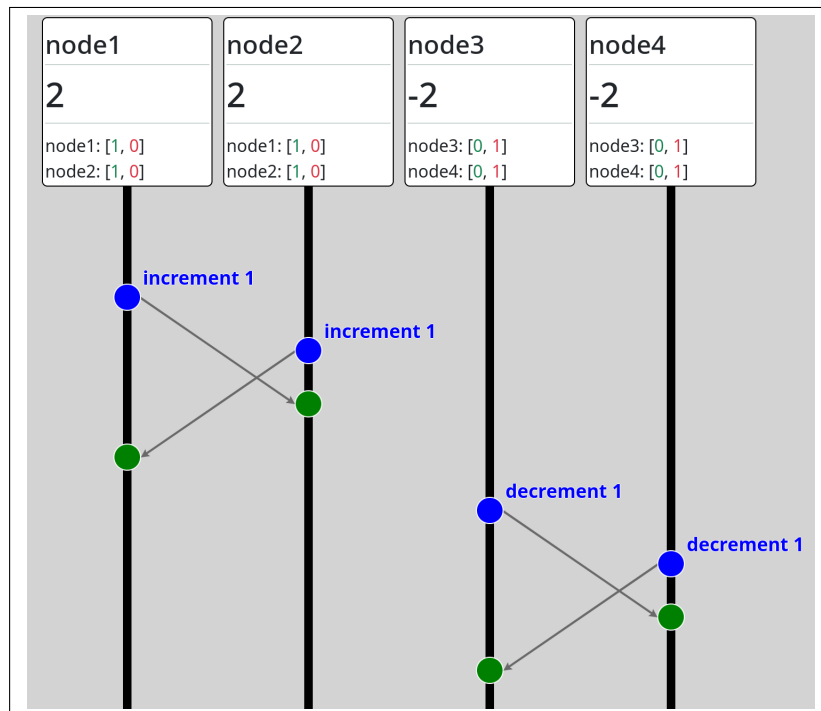
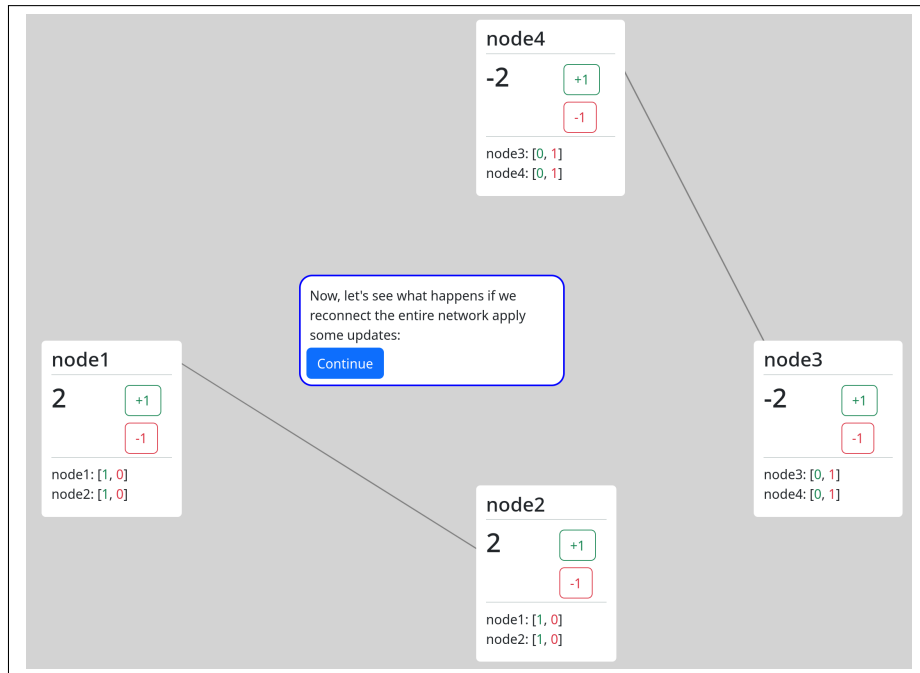


Figure A.1.: A bisected network in which both components have strongly diverged from each other.

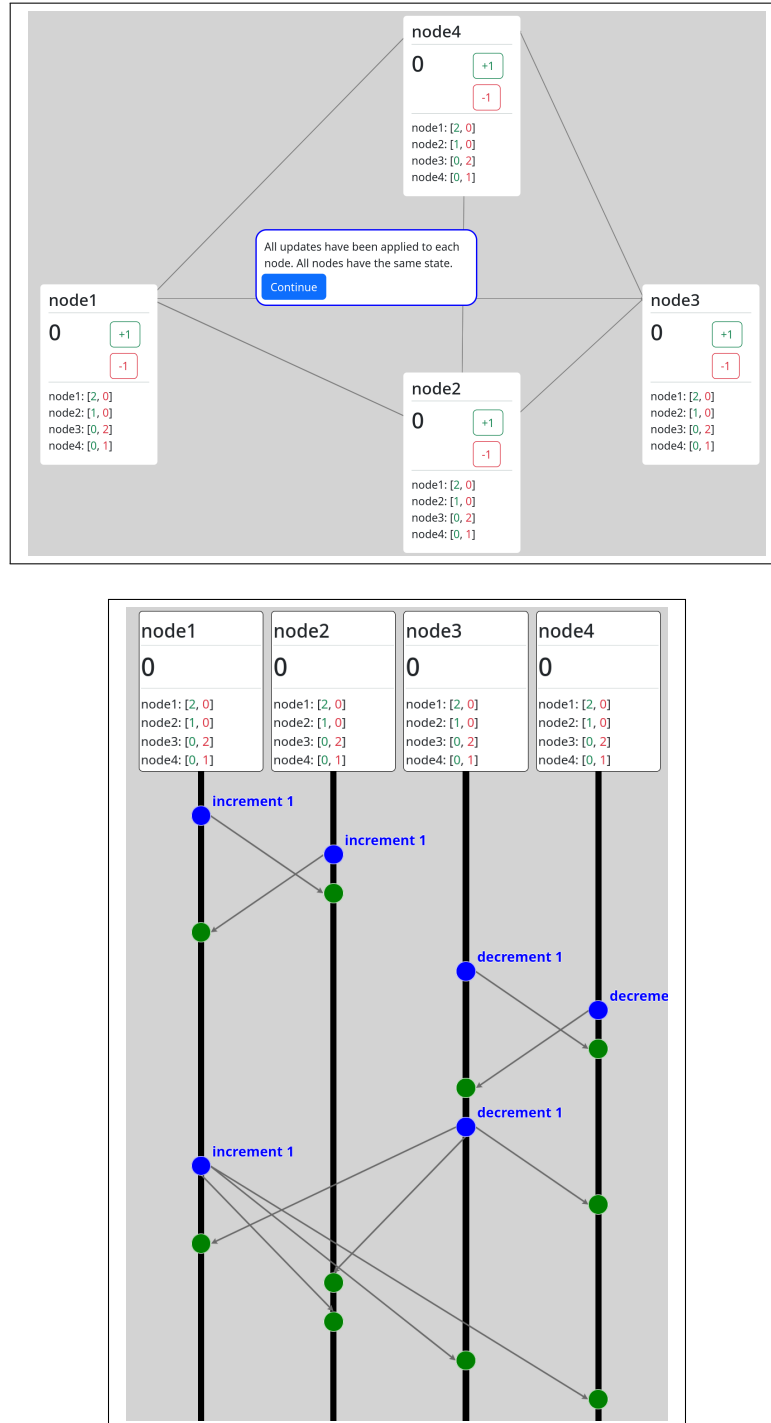


Figure A.2.: The bisected network graph has been reconnected and all nodes have registered all updates executed during the bisection period. All nodes have the same state.

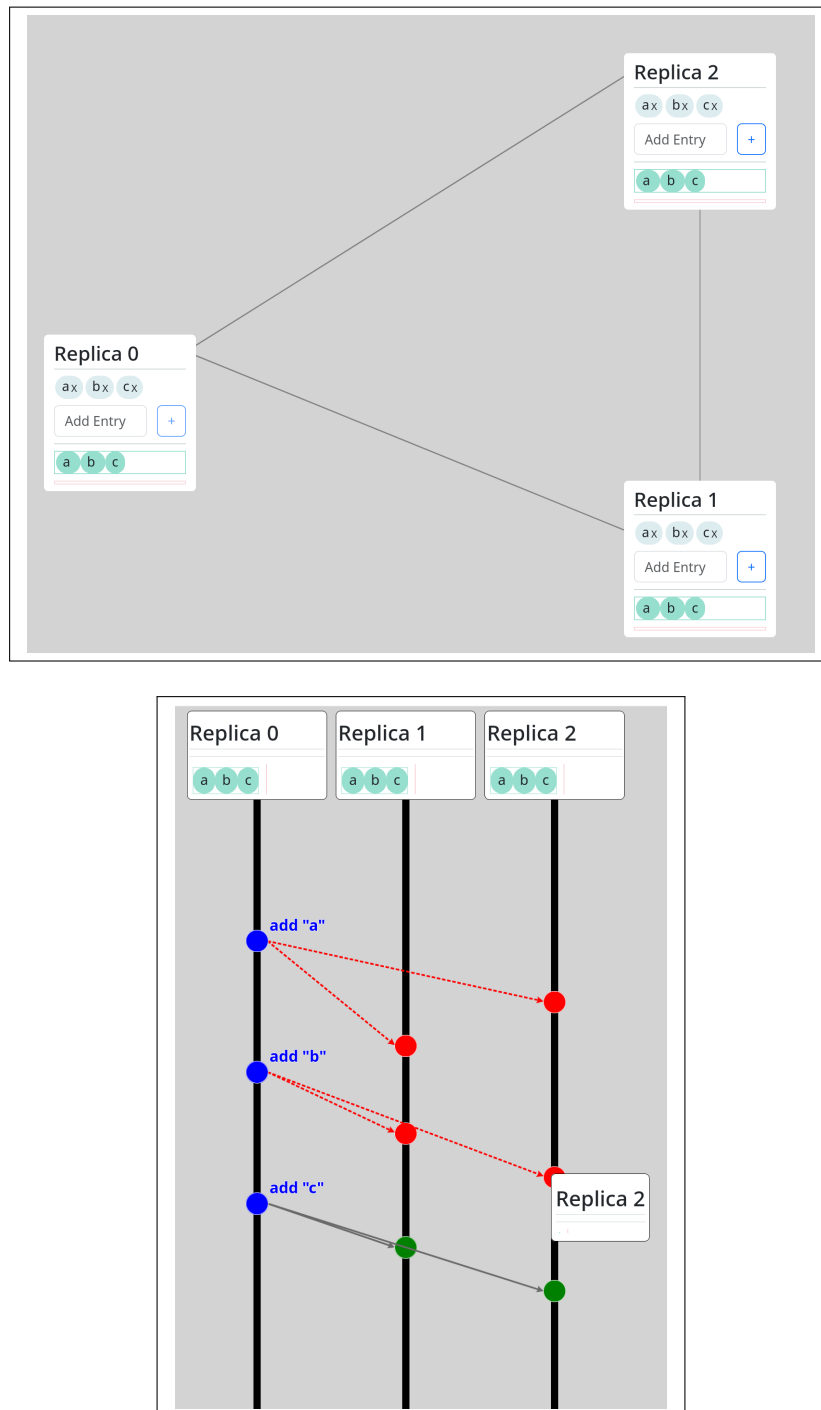


Figure A.3.: A state-based CRDT can propagate past updates despite frequent package loss.

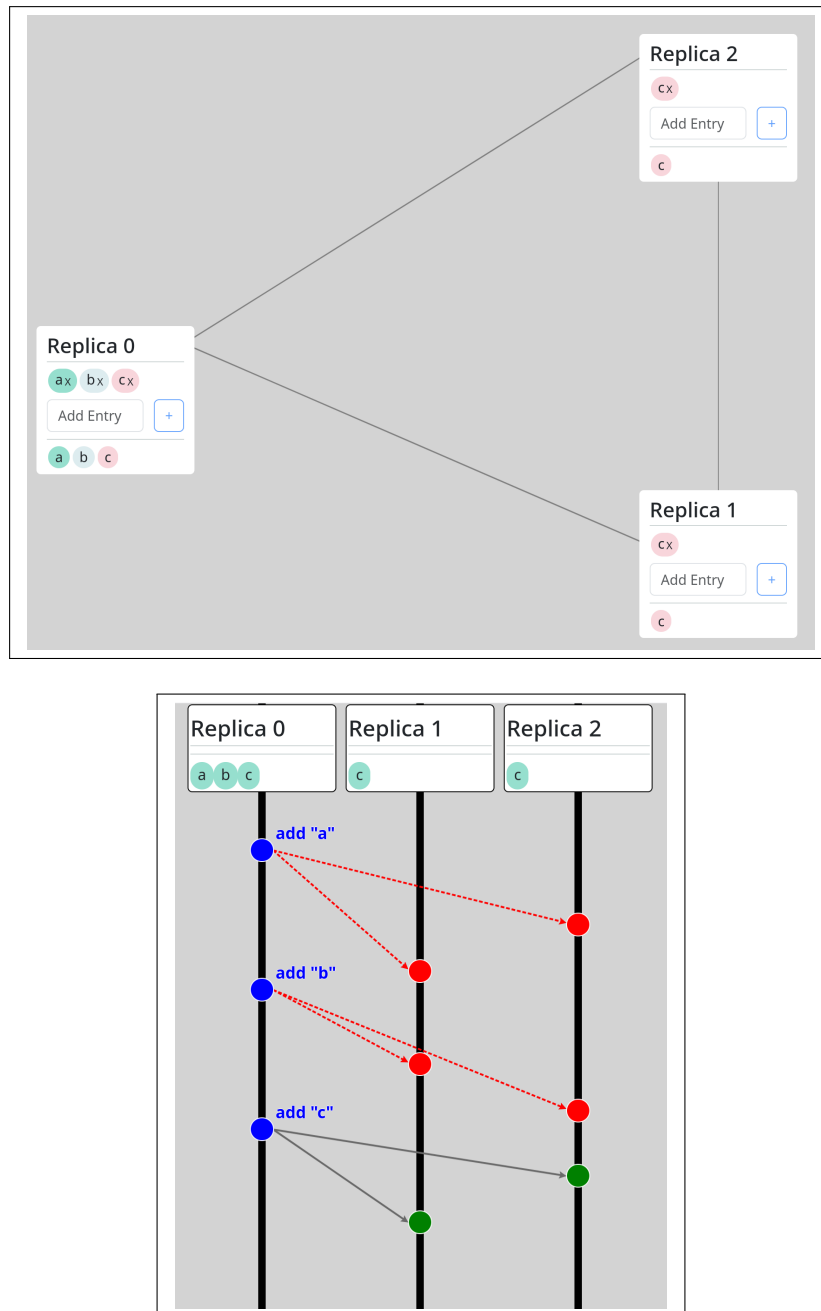


Figure A.4.: An operation-based CRDT cannot propagate past updates in a network prone to package loss without an underlying broadcast algorithm.

Bibliography

- [1] Jenny Abrahamson et al. “Shedding light on distributed system executions”. In: *Companion Proceedings of the 36th International Conference on Software Engineering*. ICSE Companion 2014. Hyderabad, India: Association for Computing Machinery, 2014, pp. 598–599. ISBN: 9781450327688. DOI: 10.1145/2591062.2591134. URL: <https://doi.org/10.1145/2591062.2591134>.
- [2] Paulo Sérgio Almeida. “Approaches to Conflict-free Replicated Data Types”. In: (Oct. 2023). eprint: 2310.18220. URL: <https://arxiv.org/abs/2310.18220>.
- [3] Paulo Sérgio Almeida, Ali Shoker, and Carlos Baquero. “Delta state replicated data types”. In: *Journal of Parallel and Distributed Computing* 111 (2018), pp. 162–173. ISSN: 0743-7315. DOI: <https://doi.org/10.1016/j.jpdc.2017.08.003>. URL: <https://www.sciencedirect.com/science/article/pii/S0743731517302332>.
- [4] anseki. *LeaderLine*. <https://github.com/anseki/leader-line>. Access date: 2025-11-28.
- [5] Ivan Beschastnikh et al. “Visualizing Distributed System Executions”. In: 29.2 (Mar. 2020). ISSN: 1049-331X. DOI: 10.1145/3375633. URL: <https://doi.org/10.1145/3375633>.
- [6] Annette Bieniusa et al. *An optimized conflict-free replicated set*. 2012. arXiv: 1210.3368 [cs.DC]. URL: <https://arxiv.org/abs/1210.3368>.
- [7] Peter Bourgon. *Roshi: a CRDT system for timestamped events*. <https://developers.soundcloud.com/blog/roshi-a-crdt-system-for-timestamped-events>. Access date: 2025-10-31. 2014.
- [8] Tim Bray. *The JavaScript Object Notation (JSON) Data Interchange Format*. RFC 8259. Dec. 2017. DOI: 10.17487/RFC8259. URL: <https://www.rfc-editor.org/info/rfc8259>.
- [9] Eric A. Brewer. “Towards robust distributed systems (abstract)”. In: *Proceedings of the Nineteenth Annual ACM Symposium on Principles of Distributed Computing*. PODC ’00. Portland, Oregon, USA: Association for Computing Machinery, 2000, p. 7. ISBN: 1581131836. DOI: 10.1145/343477.343502. URL: <https://doi.org/10.1145/343477.343502>.
- [10] Sebastian Burckhardt. “Principles of Eventual Consistency”. In: *Foundations and Trends® in Programming Languages* 1.1-2 (2014), pp. 1–150. ISSN: 2325-1107. DOI: 10.1561/25000000011. URL: <http://dx.doi.org/10.1561/25000000011>.

- [11] Sebastian Burckhardt et al. “Replicated data types: specification, verification, optimality”. In: *Proceedings of the 41st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. POPL ’14. San Diego, California, USA: Association for Computing Machinery, 2014, pp. 271–284. ISBN: 9781450325448. DOI: 10.1145/2535838.2535848. URL: <https://doi.org/10.1145/2535838.2535848>.
- [12] Liangrun Da and Martin Kleppmann. “Extending JSON CRDTs with Move Operations”. In: *11th Workshop on Principles and Practice of Consistency for Distributed Data*. PaPoC 2024. ACM, Apr. 2024, pp. 8–14. DOI: 10.1145/3642976.3653030. eprint: 2311.14007.
- [13] Xavier Défago, André Schiper, and Péter Urbán. “Total order broadcast and multicast algorithms: Taxonomy and survey”. In: *ACM Comput. Surv.* 36.4 (Dec. 2004), pp. 372–421. ISSN: 0360-0300. DOI: 10.1145/1041680.1041682. URL: <https://doi.org/10.1145/1041680.1041682>.
- [14] P. Th. Eugster et al. “Lightweight probabilistic broadcast”. In: *ACM Trans. Comput. Syst.* 21.4 (Nov. 2003), pp. 341–374. ISSN: 0734-2071. DOI: 10.1145/945506.945507. URL: <https://doi.org/10.1145/945506.945507>.
- [15] Seth Gilbert and Nancy Lynch. “Brewer’s conjecture and the feasibility of consistent, available, partition-tolerant web services”. In: *SIGACT News* 33.2 (June 2002), pp. 51–59. ISSN: 0163-5700. DOI: 10.1145/564585.564601. URL: <https://doi.org/10.1145/564585.564601>.
- [16] *ISO 25010:2023, Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality mode*. Standard. Geneva, CH: International Organization for Standardization, 2023.
- [17] Kevin Jahns. *Yjs*. <https://github.com/yjs/yjs>. Access date: 2025-11-05.
- [18] Flavio P. Junqueira, Benjamin C. Reed, and Marco Serafini. “Zab: High-performance broadcast for primary-backup systems”. In: *2011 IEEE/IFIP 41st International Conference on Dependable Systems & Networks (DSN)*. 2011, pp. 245–256. DOI: 10.1109/DSN.2011.5958223.
- [19] Leslie Lamport. “Time, clocks, and the ordering of events in a distributed system”. In: *Commun. ACM* 21.7 (July 1978), pp. 558–565. ISSN: 0001-0782. DOI: 10.1145/359545.359563. URL: <https://doi.org/10.1145/359545.359563>.
- [20] Jake Lazaroff. *An Interactive Intro to CRDTs*. <https://jakelazaroff.com/words/an-interactive-intro-to-crdts/>. Access date: 2025-10-29.
- [21] Philipp Lersch. *CRDT Showcase*. <https://softech-git.informatik.uni-kl.de/research/crdt-showcase>. Access date: 2025-11-18.
- [22] *Maintenance of duplicate databases*. RFC 677. Jan. 1975. DOI: 10.17487/RFC0677. URL: <https://www.rfc-editor.org/info/rfc677>.

- [23] Sreeja Nair et al. “A coordination-free, convergent, and safe replicated tree”. In: (Mar. 2021). eprint: 2103.04828. URL: <https://arxiv.org/abs/2103.04828>.
- [24] Francisco Neves, Nuno Machado, and José Pereira. “Falcon: A Practical Log-Based Analysis Tool for Distributed Systems”. In: *2018 48th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. 2018, pp. 534–541. DOI: 10.1109/DSN.2018.00061.
- [25] Diego Ongaro. *The Raft Consensus Algorithm*. <https://github.com/raft/raft.github.io>. Access date: 2025-10-28.
- [26] Michael Owen. *Using Erlang, Riak and the ORSWOT CRDT at bet365 for Scalability and Performance*. <https://www.erlang-factory.com/euc2015/michael-owen>. Access date: 2025-10-31. 2015.
- [27] Nuno Preguiça. “Conflict-free Replicated Data Types: An Overview”. In: (June 2018). eprint: 1806.10254. URL: <https://arxiv.org/abs/1806.10254>.
- [28] Nuno Preguiça, Carlos Baquero, and Marc Shapiro. “Conflict-Free Replicated Data Types (CRDTs)”. In: *Encyclopedia of Big Data Technologies*. Springer, May 2018. DOI: 10.1007/978-3-319-63962-8_185-1. eprint: 1805.06358.
- [29] Nuno Preguica et al. “A Commutative Replicated Data Type for Cooperative Editing”. In: *2009 29th IEEE International Conference on Distributed Computing Systems*. 2009, pp. 395–403. DOI: 10.1109/ICDCS.2009.20.
- [30] Hyun-Gul Roh et al. “Replicated abstract data types: Building blocks for collaborative applications”. In: *Journal of Parallel and Distributed Computing* 71.3 (2011), pp. 354–368. ISSN: 0743-7315. DOI: <https://doi.org/10.1016/j.jpdc.2010.12.006>. URL: <https://www.sciencedirect.com/science/article/pii/S0743731510002716>.
- [31] High Scalability. *How League of Legends Scaled Chat to 70 million Players - It takes Lots of minions*. <https://highscalability.com/how-league-of-legends-scaled-chat-to-70-million-players-it-t/>. Access date: 2025-10-31. 2014.
- [32] Nicolas Seriot. *iOS-Runtime-Headers*. <https://github.com/nst/iOS-Runtime-Headers/blob/master/PrivateFrameworks/NotesShared.framework/TTMergeableString.h>. Access date: 2025-10-31. 2018.
- [33] Marc Shapiro et al. *A comprehensive study of Convergent and Commutative Replicated Data Types*. Research Report RR-7506. Inria – Centre Paris-Rocquencourt ; INRIA, Jan. 2011, p. 50. URL: <https://inria.hal.science/inria-00555588>.

- [34] Xiao Shi et al. “FlightTracker: Consistency across Read-Optimized On-line Stores at Facebook”. In: *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. USENIX Association, Nov. 2020, pp. 407–423. ISBN: 978-1-939133-19-9. URL: <https://www.usenix.org/conference/osdi20/presentation/shi>.
- [35] Guy L. Steele. *Common LISP: the language (2nd ed.)* USA: Digital Press, 1990. ISBN: 1555580416.
- [36] Paolo Viotti and Marko Vukolić. *Consistency in Non-Transactional Distributed Storage Systems*. 2016. arXiv: 1512.00168 [cs.DC]. URL: <https://arxiv.org/abs/1512.00168>.
- [37] Marek Zawirski et al. *Eventually Consistent Register Revisited*. 2015. arXiv: 1511.05010 [cs.DC]. URL: <https://arxiv.org/abs/1511.05010>.